GROUP ID: _____

A PROJECT REPORT ON
GAIA: A ROVER-ASSISTED GENERATIVE AI SYSTEM FOR PROACTIVE CROP HEALTH MONITORING

SUBMITTED TO THE PIMPRI CHINCHWAD COLLEGE OF ENGINEERING AN AUTONOMOUS INSTITUTE, PUNE

IN THE FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE

OF

B. TECH. IN "COMPUTER SCIENCE AND ENGINEERING (AI & ML)"

FOR THE SUBJECT

GENERATIVE AI MDM CAPSTONE PROJECT
UNDER THE DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

SUBMITTED BY

STUDENT NAME 1 PRN: _____
STUDENT NAME 2 PRN: _____
STUDENT NAME 3 PRN: _____
STUDENT NAME 4 PRN: _____

UNDER THE GUIDANCE OF

GUIDE NAME

Insert image: PCCOE logo

figures/pccoe_logo.png

DEPARTMENT OF COMPUTER ENGINEERING

PCET'S PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

Sector No. 26, Pradhikaran, Nigdi, Pimpri-Chinchwad, PUNE – 411044

2026-2027

PCET'S PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
Sector No. 26, Pradhikaran, Nigdi, Pimpri-Chinchwad, PUNE 411044

2026-2027

Insert image: PCCOE logo

figures/pccoe_logo.png

CERTIFICATE

This is to certify that the project report entitled
"GAIA: A ROVER-ASSISTED GENERATIVE AI SYSTEM FOR PROACTIVE CROP HEALTH MONITORING"

Submitted by

STUDENT NAME 1 PRN: _____
STUDENT NAME 2 PRN: _____
STUDENT NAME 3 PRN: _____
STUDENT NAME 4 PRN: _____

are bonafide students of this institute and the work has been carried out by them under the supervision of **Prof. GUIDE NAME** and it is approved for the partial fulfillment of the requirement of Pimpri Chinchwad College of Engineering, an autonomous institute, for the award of the B. Tech. degree in "Computer Science and Engineering (AI & ML)".

(Prof. GUIDE NAME)
Guide
Department of CSE (AI & ML)

(Prof. Dr. Anuradha Thakare)
Head,
Department of CSE (AI & ML)

Place : Pune
Date : _____

ACKNOWLEDGEMENT

It gives us great pleasure to present this project report on *GAIA: A Rover-Assisted Generative AI System for Proactive Crop Health Monitoring*. The work could not have been completed without the guidance, encouragement and support of several people, and we take this opportunity to thank them.

We express our sincere gratitude to our project guide, Prof. GUIDE NAME, for the continuous guidance, constructive criticism and the freedom to take the design decisions described in this report. The regular reviews pushed us to measure what we built instead of only describing it.

We are thankful to **Prof. Dr. Anuradha Thakare**, Head of the Department of Computer Science and Engineering (AI & ML), for providing the laboratory facilities and a supportive academic environment. We also thank the faculty members of the department for their valuable suggestions during the project reviews, and the Principal and management of Pimpri Chinchwad College of Engineering, Pune, for the opportunity to carry out this capstone project.

We acknowledge the open-source communities behind PostgreSQL, pgvector, FastAPI, Flutter and the Arduino and ESP32 toolchains, and the researchers whose published work is cited in the references. The sample crop photographs used in the demonstration were taken from Wikimedia Commons under open licences, and are credited in the project repository.

Finally, we thank our families and friends for their patience and encouragement throughout the project.

STUDENT NAME 1
STUDENT NAME 2
STUDENT NAME 3
STUDENT NAME 4

ABSTRACT

Crop-protection tools are mostly reactive: they work only after a farmer has noticed a symptom and photographed it. This project presents **GAIA**, a rover-assisted Generative AI system for proactive crop health monitoring. A small rover (ESP32-CAM, Arduino Nano, DHT11, MQ-5, MQ-6 and a servo-mounted soil-moisture probe) is only a thin data-acquisition layer. The effort is concentrated in two independent Generative AI pipelines that share one knowledge store (PostgreSQL with pgvector) and are delivered through a Flutter application and a Python (FastAPI) backend.

Pipeline A is a conversational assistant. A language model turns a question into a structured query plan, SQL computes exact statistics, a hybrid retriever (vector and full-text search fused by Reciprocal Rank Fusion) returns relevant chunks, and Gemini writes a streamed answer whose citations are validated against the retrieved context. *Pipeline B* diagnoses crops: one vision-language call turns a photograph into strict JSON, and one research agent searches the web for care or treatment information; the result is shown with sources and written one way into the knowledge store.

To save battery, every sensor can be switched on or off. The backend records switches, readings and rover heartbeats, so data gaps are explained by code ("MQ5 was switched off from 06:00 to 09:20") and not guessed by the model. A Live view animates readings arriving, windows closing, and chunks being written, embedded and stored.

The system was verified with 96 backend and 22 application tests and by compiling both firmware sketches. On a deterministic synthetic farm, planner accuracy, numeric recall and citation accuracy were all 100 % on ten questions. The backend ingested about 5,000 sensor values per second and indexed a 15-minute window in about 73 ms. Model confidence is shown as an uncalibrated indicator, and diagnoses are phrased as possibilities.

Keywords: Generative AI, Retrieval-Augmented Generation, Vision-Language Model, Agriculture, IoT, pgvector, Gemini, Flutter.

TABLE OF CONTENTS

Acknowledgement **iii**

Abstract **iv**

List of Abbreviations **viii**

Nomenclature **x**

1 Introduction **1**

1.1 Overview of Generative AI 1

1.2 Project Background 2

1.3 Motivation 3

1.4 Problem Statement 4

1.5 Objectives 4

1.6 Scope of the Work 5

2 Literature Survey **7**

2.1 Review of Recent Literature 7

2.2 Generative AI: Existing Approaches 9

2.3 Review of Related Generative AI Systems 11

2.4 Existing Solutions / State-of-the-Art Methods 11

2.5 Gap Identification 11

2.6 Common Findings from Literature 12

3 Requirements Specification **13**

3.1 Hardware Requirements 13

3.2 Software Requirements 16

3.3 Functional Requirements 17

3.4 Non-Functional Requirements 18

3.5 Generative AI Model Requirements 19

3.6 Dataset / Knowledge-Base Requirements 19

4 System Design **21**

4.1 Proposed System Architecture 21

4.2 System Block Diagram 23

4.3 System Workflow 25

4.4 Mathematical Model 30

5 Project Implementation **33**

5.1 Overview of Project Modules 33

5.2 Data Collection and Preprocessing 35

5.3 Generative AI Model Selection 36

5.4 Model Configuration / Fine-Tuning 37

5.5 Project Modules 38

5.6 Tools and Technologies Used 47

5.7 Algorithm Details 47

6 Software Testing **49**

6.1 Test Cases 49

6.2 Generative AI Output Testing 52

6.3 Performance Evaluation 55

7 Results and Discussion **58**

7.1 Project Outcomes 58

7.2 Result Analysis 58

7.3 Generated Output Analysis 60

7.4 Performance Metrics 61

7.5 Accuracy / Relevance / Quality Evaluation 61

7.6 Validation of Results 62

7.7 Comparison with Existing Approaches 63

7.8 Screenshots of the Implemented System 64

8 Conclusion and Future Work **70**

8.1 Conclusion 70

8.2 Limitations of the Proposed System 71

8.3 Future Work 72

8.4 Applications 73

References **74**

Appendix A: Paper Publication Details **77**

Appendix B: Plagiarism Report **78**

Appendix C: REST API Summary **79**

LIST OF ABBREVIATIONS

ABBREVIATION	ILLUSTRATION
ADC	Analog-to-Digital Converter
AI	Artificial Intelligence
API	Application Programming Interface
APK	Android Package (installable Android application)
CI/CD	Continuous Integration / Continuous Delivery
CORS	Cross-Origin Resource Sharing
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
DHT11	Digital Humidity and Temperature sensor, model 11
ECE	Expected Calibration Error
ER	Entity–Relationship
ESP32-CAM	ESP32 microcontroller module with a camera
FR / NFR	Functional Requirement / Non-Functional Requirement
FTS	Full-Text Search
GenAI	Generative Artificial Intelligence
GIN	Generalized Inverted Index (PostgreSQL)
GPIO	General-Purpose Input/Output
HNSW	Hierarchical Navigable Small World (approximate nearest-neighbour index)
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
JPEG	Joint Photographic Experts Group (image format)
JSON	JavaScript Object Notation
LLM	Large Language Model
MJPEG	Motion JPEG (video as a stream of JPEG frames)
MQ-5, MQ-6	Metal-oxide gas sensors (natural gas / methane, and LPG / butane)
NTP	Network Time Protocol
ORM	Object-Relational Mapping
PDF	Portable Document Format
PRN	Permanent Registration Number
PWM	Pulse-Width Modulation
RAG	Retrieval-Augmented Generation

ABBREVIATION	ILLUSTRATION
REST	Representational State Transfer
RRF	Reciprocal Rank Fusion
RTC	Real-Time Clock
SQL	Structured Query Language
SSE	Server-Sent Events
SSID	Service Set Identifier (Wi-Fi network name)
UART	Universal Asynchronous Receiver/Transmitter
UML	Unified Modeling Language
URL	Uniform Resource Locator
UTC	Coordinated Universal Time
VLM	Vision-Language Model

NOMENCLATURE

Symbols and notations used in Section 4.4 and elsewhere in this report are listed here, and only here.

SYMBOL	MEANING
s	A sensor channel, for example temperature, humidity, soil moisture, MQ-5 or MQ-6
$x_{s,i}$	The i-th reading of sensor s
Δ	Aggregation window length (15 minutes by default)
W_k	The k-th window, $[t_k, t_k + \Delta)$
$n_{s,k}$	Number of readings of sensor s inside window W_k
$\mu_{s,k},\ \sigma_{s,k}$	Mean and standard deviation of sensor s in window W_k
$m_{s,k},\ M_{s,k}$	Minimum and maximum of sensor s in window W_k
$\mu_s(R)$	Count-weighted mean of sensor s over a set of windows R
t_i^{recv}	Time at which the backend received a batch
$u_i,\ u^{\text{sent}}$	Rover uptime at reading i, and at the moment the batch was sent (milliseconds)
$\hat{t}_i$	Reconstructed wall-clock time of reading i
$\mathbf{q},\ \mathbf{d}$	Unit-length embedding vectors of a query and of a document chunk
$\cos(\mathbf{q}, \mathbf{d})$	Cosine similarity of $\mathbf{q}$ and $\mathbf{d}$
$\text{rank}_r(d)$	Rank of chunk d in ranked list r (1 is best)
k	Reciprocal Rank Fusion constant ($k = 60$)
$\text{RRF}(d)$	Fused score of chunk d
b	A five-minute time bucket
$\text{st}_s(b)$	Status of sensor s in bucket b (recorded, off, no reading, offline, unknown)
T_w	Heater warm-up time of an MQ sensor (60 s)
$p_i,\ y_i$	Self-reported confidence of the i-th diagnosis, and whether it was correct (1) or not (0)
B_j	The j-th confidence bin used for calibration
ECE	Expected Calibration Error
BS	Brier score
$R_1,\ R_2$	Resistors of the battery voltage divider
V_{bat}	Battery voltage
$P_h,\ t_{\text{off}}$	Heater power and the time a sensor is switched off

LIST OF FIGURES

1.1 System overview: the rover feeds ingestion, Pipeline B writes one way into the shared knowledge store, and Pipeline A reads from it. 2

1.2 Two independent Generative AI pipelines sharing a single knowledge store. Pipeline B never reads sensor data and Pipeline A never sees image pixels. 3

1.3 Project timeline (weeks). 6

3.1 Hardware block diagram of the rover. 15

4.1 Layered architecture of GAIA. 21

4.2 Deployment view: devices, ports and protocols. 22

4.3 System block diagram with the three data paths. 23

4.4 Use-case diagram. 24

4.5 Entity–relationship diagram of the database. 25

4.6 Lifecycle of a sensor reading. 26

4.7 Workflow of Pipeline A (“Query my farm”). 27

4.8 Workflow of Pipeline B (crop diagnosis). 28

4.9 State machine of a diagnosis job. 28

4.10 Sequence diagram of a streamed “Query my farm” answer. 29

4.11 Status of a sensor in a five-minute bucket. 30

7.1 Mobile application: overview and rover control. 64

7.2 Mobile application: diagnosis and chat. 65

7.3 Mobile application: sensor values and a cited chat answer. 65

7.4 Web interface: a diagnosis result with evidence, confidence caveat and sources. 66

7.5 Web interface: knowledge-store inspector showing vector, lexical and fused scores. 67

7.6 Mobile application: Live view and per-sensor switches. 67

7.7 Mobile application: streamed answer and gap explanation. 68

7.8 The assembled rover. 68

7.9 Soil-probe arm and field trial. 69

LIST OF TABLES

1.1 Project plan (indicative). 6

2.1 Summary of the reviewed literature and its relevance to GAIA. 9

2.2 Approaches to adapting a generative model to a domain. 10

2.3 Related systems compared with GAIA, based on the cited publications. A dash means the publication does not report the feature and no claim is made. 11

3.1 Hardware components. 14

3.2 Software and versions used. 16

3.3 Functional requirements. 17

3.4 Non-functional requirements. 18

3.5 Model roles and requirements. 19

5.1 Project modules and their locations. 34

5.2 Main backend settings (`backend/.env`) and defaults. 37

5.3 HTTP API of the ESP32-CAM. 39

5.4 Tools and technologies. 47

5.5 Algorithm summary. 48

6.1 Automated tests by area (all passing). 50

6.2 Selected functional test cases. 50

6.3 The ten evaluation questions (intent and expected evidence). 54

6.4 Measured backend performance. 56

7.1 Pipeline A evaluation on the synthetic farm (10 questions). 59

7.2 Headline performance and resource figures. 61

7.3 Vision step on 60 PlantVillage photographs (to be completed). 62

7.4 Feature comparison (a dash means not claimed). 63

8.1 Objectives and outcomes. 71

8.2 Publication details (to be filled). 77

8.3 Backend endpoints. 79

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW OF GENERATIVE AI

Generative Artificial Intelligence (GenAI) refers to models that produce new content, such as text, images or structured data, instead of only assigning a label to an input. Most of the current systems are built on the Transformer architecture [1]. Large Language Models (LLMs) trained on very large text collections were shown to perform many tasks from a handful of examples in the prompt [2], and multimodal models such as Gemini accept text, images and other media in one request [3, 4]. For an application developer, three capabilities of these models matter most.

- **Understanding free-form language and images.** A farmer can ask "Was the soil drier than yesterday?" in ordinary words, or send a photograph of a leaf, without choosing from a menu.
- **Structured output.** A model can be instructed to return JSON that follows a schema. This lets deterministic program code, and not the model, decide what happens next.
- **Tool use and retrieval.** A model can be given retrieved documents or web search results and asked to answer from them. This approach is called Retrieval-Augmented Generation (RAG) [5, 6], and a loop in which the model decides what to look up next is the basis of "agentic" systems [7, 8].

The same models also have well-known weaknesses. They can state false information fluently, a behaviour known as hallucination [9], and their own statements of confidence are not reliable probabilities [10, 11]. A responsible application therefore has to decide which parts of an answer may come from a model (wording, explanation) and which parts must come from verifiable code (numbers, dates, citations). This division is the central design principle of the project described in this report.

1.2 PROJECT BACKGROUND

Crop losses often begin as small changes that are only noticed late: a few spotted leaves, a humid spell that favours fungal disease, or soil that has been drying for days. Two kinds of tools address this today. Mobile applications for plant disease identification, such as Plantix [12], are *reactive*: they help only after the farmer has noticed a problem and photographed it. Sensor kits log temperature, humidity or soil moisture, but their output is a table or a chart that the farmer has to interpret.

GAIA (the project name) combines the two ideas around a small, low-cost *rover*. The rover carries an ESP32-CAM, an Arduino Nano, a DHT11 temperature and humidity sensor, MQ-5 and MQ-6 gas sensors and a soil-moisture probe on a servo-controlled arm. It streams live video to a Flutter mobile application and records field measurements. The rover is deliberately kept as a thin data-acquisition layer. The substance of the project lies in two independent Generative AI pipelines that run on a Python backend (Figure 1.1):

- **Pipeline A, the conversational assistant.** In "Query my farm" mode it answers questions from the farmer's own sensor log, earlier diagnoses and notes. In "Open-ended" mode it answers general questions with web search.
- **Pipeline B, crop diagnosis.** A single vision-language model (VLM) call analyses a photograph and one research agent looks up care or treatment information on the web.

The two pipelines share one knowledge store (PostgreSQL with the pgvector extension) but have no control-flow dependency on each other (Figure 1.2).

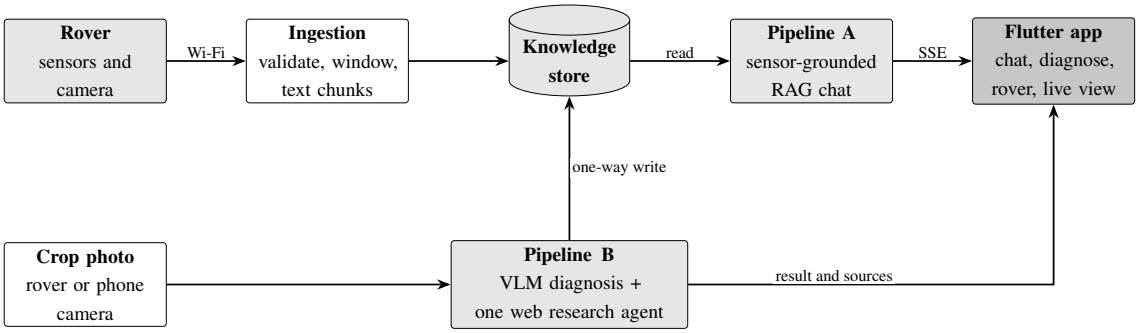


Figure 1.1: System overview: the rover feeds ingestion, Pipeline B writes one way into the shared knowledge store, and Pipeline A reads from it.

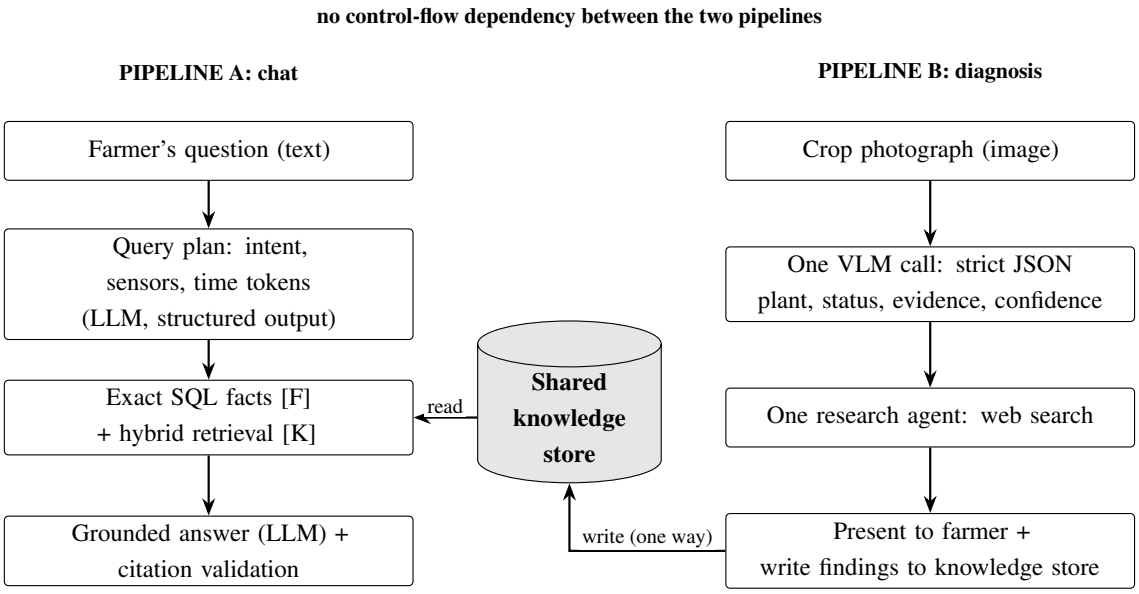


Figure 1.2: Two independent Generative AI pipelines sharing a single knowledge store. Pipeline B never reads sensor data and Pipeline A never sees image pixels.

1.3 MOTIVATION

1. **Data is collected but not understood.** A farmer who owns a sensor cannot easily ask "how humid was my field this morning?" or "were there unusual gas readings in the last three days?" without reading charts. A language interface removes this barrier, but only if the answers are grounded in the real measurements.
2. **Generic chatbots do not know the farm.** A general LLM can describe early blight but cannot know that the humidity in the north plot stayed above 85 % all night. Retrieval over the farmer's own data closes this gap.
3. **Photo diagnosis needs honesty.** Recent benchmarks show that vision-language models are good at the coarse healthy-or-diseased decision but much weaker at naming the exact pathogen [13, 14]. A diagnosis tool therefore has to present results as possibilities, with evidence and an explicit statement of uncertainty.
4. **The rover runs on a battery.** Gas-sensor heaters draw far more current than the rest of the sensor payload. Allowing the farmer to switch each sensor on or off saves energy, but it creates gaps in the data. The AI must then explain these gaps correctly instead of inventing values.
5. **Academic requirement.** The capstone evaluation asks for a genuine and complex Generative AI application with responsible-AI practices, not primarily a robotics or IoT project.

1.4 PROBLEM STATEMENT

To design, implement and evaluate a rover-assisted Generative AI system that (i) continuously collects field measurements and turns them into a retrievable knowledge base, (ii) answers farmers' natural-language questions about their own field with exact numbers and verifiable citations, (iii) analyses crop photographs with a vision-language model and retrieves web-based care or treatment information, (iv) remains truthful when data is missing because a sensor was switched off or the rover was disconnected, and (v) is delivered as a mobile application with a streaming, low-latency user experience, while disclosing the limits of the underlying models.

1.5 OBJECTIVES

O1. Build a rover data-acquisition layer that streams video, captures high-resolution stills and reports temperature, humidity, gas and soil-moisture readings over Wi-Fi.

O2. Implement a validated, timestamped ingestion path with deterministic aggregation into fixed 15-minute windows and day summaries, written as natural-language text and embedded into a vector database.

O3. Implement Pipeline A: a query planner, exact SQL statistics, hybrid vector and full-text retrieval with Reciprocal Rank Fusion, grounded generation, and citation validation.

O4. Implement Pipeline B: one vision-language model call that returns strict JSON, one web research agent, and a one-way write of the findings into the shared knowledge store.

O5. Add per-sensor on/off switches for battery saving, and make the AI explain data gaps from deterministic records.

O6. Deliver a Flutter application with rover control, sensor view, streamed chat, diagnosis, and a Live view that shows the pipeline working.

O7. Apply responsible-AI practices: uncalibrated-confidence disclosure, possibility-phrased diagnoses, hallucinated-citation removal, and chemical-safety rules.

O8. Verify the system with automated tests and a quantitative evaluation on a deterministic synthetic farm, and measure its performance.

1.6 SCOPE OF THE WORK

1.6.1 In Scope

- One rover, one farm and one farmer account; no user-account system.
- Rover firmware for the ESP32-CAM and the Arduino Nano, the HTTP API of the rover, and the UART link between them.
- The Python backend: ingestion, knowledge store, both pipelines, the live activity stream, a small web interface, the rover simulator and the evaluation scripts.
- The Flutter mobile application (Android build produced by a GitHub Actions workflow).
- Gemini as the default provider for chat, vision and embeddings, with OpenAI-compatible providers (Groq, xAI Grok, local Ollama) as alternatives selected by configuration.

1.6.2 Out of Scope

- Training or fine-tuning a model. All models are used through their APIs with prompts and schemas.
- Rover localisation (GPS or odometry) and autonomous navigation.
- Calibration of the MQ gas sensors to parts-per-million values. They are reported as relative indices.
- Regulatory certification of agronomic advice. Output is guidance that the farmer should confirm locally.
- Multi-tenant security, offline-first mobile synchronisation, and regional-language voice interaction. These are listed as future work in Chapter 8.

1.6.3 Development Approach and Timeline

The project followed an incremental (agile) approach: a working end-to-end slice (rover simulator, ingestion, chat) was built first, and each later feature (diagnosis, streaming, sensor switches, Live view) was added and tested on top of it. Table 1.1 summarises the plan and Figure 1.3 shows it on a timeline. Adjust the weeks to your actual schedule.

Table 1.1: Project plan (indicative).

No.	Activity	Weeks	Output
1	Literature survey and requirements	1–3	Requirement list
2	Rover hardware and firmware	2–6	Working rover API
3	Backend: ingestion and knowledge store	3–7	Searchable sensor log
4	Pipeline A: sensor-grounded RAG chat	5–9	Cited answers
5	Pipeline B: VLM diagnosis and agent	6–10	Diagnosis with sources
6	Flutter application	7–12	Installable APK
7	Sensor switches, streaming, Live view	11–14	Gap-aware answers
8	Testing and evaluation	12–15	Test and metric reports
9	Documentation and report	13–16	This report

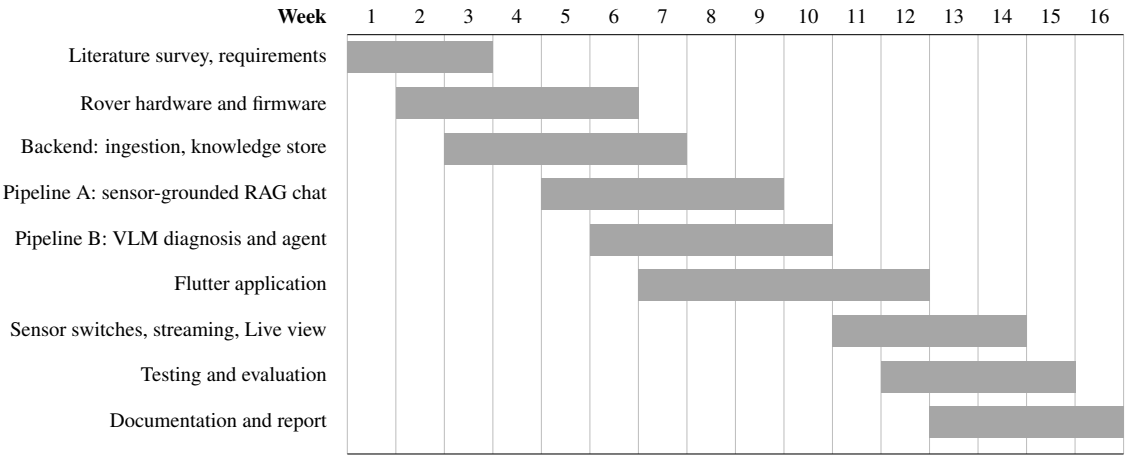


Figure 1.3: Project timeline (weeks).

CHAPTER 2

LITERATURE SURVEY

2.1 REVIEW OF RECENT LITERATURE

2.1.1 Machine Learning in Agriculture

Machine learning in agriculture was reviewed in broad terms by Liakos et al. [15], who grouped the work by application: crop management, livestock, water and soil management. Kamilaris and Prenafeta-Boldú [16] surveyed deep learning in agriculture and found that convolutional networks dominated image-based tasks such as disease detection and weed identification. For plant disease detection in particular, Mohanty et al. trained deep networks on the PlantVillage image collection and reported high accuracy on held-out images from the same collection [17, 18], and Ferentinos [19] compared several architectures on a larger set of leaf images. These works establish that supervised image classifiers can be accurate when images resemble the training data (single leaves on plain backgrounds), but they output a class label only: no explanation, no treatment advice and no link to the conditions in the field.

2.1.2 Large Language Models for Agricultural Question Answering

Silva et al. examined whether a general-purpose model, GPT-4, can answer agriculture examination questions [20]. Balaguer et al. compared retrieval-augmented generation with fine-tuning in a pipeline study that used agriculture as its case study, and reported that adaptation to local, geography-specific knowledge improved the similarity of answers to reference answers [21]. Yang et al. presented ShizishanGPT, an agricultural assistant that combines a general model with a search engine, a knowledge graph, a retrieval module and an agent that can call specialised crop models [22]. Together these studies show that retrieval and tool use are the accepted way to bring domain and local knowledge into a general model. They also share a limitation relevant to this project: the knowledge comes from documents, benchmarks and public sources, not from the *farmer's own live measurements*.

2.1.3 Vision-Language Models for Plant Disease

Two recent benchmarks evaluate general vision-language models on agricultural images. Ranario and Earles benchmarked open and closed models on many agricultural classification datasets and concluded that, used zero-shot, the models trail a supervised baseline and perform much worse with open-ended prompting than with constrained, multiple-choice questions. They recommend using such models with constrained interfaces and not as stand-alone diagnostic tools [14]. LeafNet introduced a large visual question-answering benchmark for plant diseases and reported that models do well on the coarse healthy-versus-diseased decision but considerably worse when asked to identify the pathogen or the species [13]. These findings directly shaped the design of Pipeline B: a strict JSON schema constrains the model, the output separates observed evidence from assumptions, the diagnosis is phrased as a possibility, and a list of look-alike conditions is requested.

2.1.4 Confidence and Uncertainty

Neural classifiers are often over-confident, and calibration is commonly measured with the expected calibration error and reliability diagrams [23]. Language models can be asked to state their confidence, and studies show that these verbalised numbers carry some signal but are imperfect [10, 11]. For this reason GAIA never presents the model's self-reported confidence as a probability: it is shown as a rough indicator with a fixed caveat, and the evaluation scripts compute the calibration metrics so that the caveat is backed by measurements.

Table 2.1: Summary of the reviewed literature and its relevance to GAIA.

Work	Focus	Relevance to GAIA
Liakos et al. [15]; Kamilaris and Prenafeta-Boldú [16]	Reviews of machine learning and deep learning in agriculture	Establish the application area; image classifiers dominate but give labels only
Mohanty et al. [17]; Ferentinos [19]	CNN disease classifiers on leaf images	PlantVillage is used in GAIA's vision benchmark script; results are an upper bound for field photos
Silva et al. [20]	GPT-4 on agriculture exams	Motivates grounding a general model in domain data
Balaguer et al. [21]	RAG versus fine-tuning, agriculture case study	Supports choosing RAG with prompts over fine-tuning
Yang et al. [22]	Agricultural LLM with search, knowledge graph, RAG and agent	Closest system: tool-using assistant, but no live farm sensor data
Ranario and Earles [14]	Zero-shot VLMs versus supervised models	Motivates constrained, strict-JSON prompting and honest framing
Nguyen Quoc et al. [13]	Plant-disease VQA benchmark	Coarse health is reliable, fine identification is not: show possibilities and differentials
Guo et al. [23]; Xiong et al. [11]	Calibration and confidence elicitation	Justify the uncalibrated-confidence caveat and the ECE and Brier metrics

2.2 GENERATIVE AI: EXISTING APPROACHES

2.2.1 Prompting, Retrieval, Fine-Tuning and Agents

Four approaches are commonly used to adapt a general model to a domain. Table 2.2 compares them for the needs of this project.

Table 2.2: Approaches to adapting a generative model to a domain.

Approach	Strengths	Weaknesses for GAIA	Used?
Plain prompting	Simple, cheap	Cannot know the farm's data	Only for small talk
Retrieval-augmented generation [5]	Uses fresh private data; answers can cite sources	Needs good retrieval; model can still ignore context	Yes (Pipeline A)
Fine-tuning	Learns style or domain vocabulary	Cannot absorb live sensor data; needs data and compute	No
Agents with tools [7]	Can decide what to look up	Unbounded loops, cost, harder to test	Yes, bounded (Pipeline B)

2.2.2 Retrieval Techniques

Dense retrieval embeds text into vectors and finds nearest neighbours by cosine similarity. Approximate nearest-neighbour indexes such as Hierarchical Navigable Small World (HNSW) graphs make this fast for large collections [24], and the pgvector extension provides them inside PostgreSQL [25]. Dense retrieval captures meaning ("dry soil" is close to "low soil moisture") but can miss exact tokens such as sensor names. Lexical search has the opposite profile. Combining the two rankings with Reciprocal Rank Fusion (RRF) is a simple, parameter-light method that was shown to outperform several learned fusion methods [26]. GAIA uses this hybrid, with the Gemini embedding model providing the vectors [27].

2.2.3 Multimodal Models

Contrastive pre-training on image–text pairs [28] and visual instruction tuning [29] led to models that can describe and reason about images. Commercial multimodal models such as Gemini accept an image and a text instruction together and can return structured output [3, 4, 30]. GAIA uses this capability in a single call to analyse a crop photograph.

2.3 REVIEW OF RELATED GENERATIVE AI SYSTEMS

Table 2.3: Related systems compared with GAIA, based on the cited publications. A dash means the publication does not report the feature and no claim is made.

System	Image diagnosis	Language Q&A	Web or document retrieval	Own live sensor data	Explains data gaps
Plantix [12]	Yes	–	–	No	No
CNN classifiers [17, 19]	Yes (label)	No	No	No	No
GPT-4 on exams [20]	No	Yes	–	No	No
ShizishanGPT [22]	Not reported	Yes	Yes	No	No
GAIA (this work)	**Yes (VLM)**	**Yes**	**Yes**	**Yes (rover)**	**Yes**

2.4 EXISTING SOLUTIONS / STATE-OF-THE-ART METHODS

- **Consumer disease-identification applications.** Photograph in, label and advice out [12]. They depend on the farmer noticing a symptom.
- **Supervised leaf classifiers.** High accuracy on curated datasets [17, 19] but little transfer to field photographs and no explanation.
- **Retrieval-augmented agricultural assistants.** Document-grounded answers, sometimes with tools and knowledge graphs [21, 22].
- **General multimodal models used zero-shot.** Flexible, but weaker than supervised models on fine-grained agricultural classes [14, 13].

2.5 GAP IDENTIFICATION

G1. **Reactive operation.** Existing diagnosis tools wait for the farmer to notice a problem. A rover that walks the field and measures conditions on demand supports proactive inspection.

G2. **No grounding in the farmer's own measurements.** Retrieval-based agricultural assistants answer from public or curated documents. None of the reviewed systems

answers from the farm's own sensor log with exact statistics.

G3. **Numbers from language models.** Letting a model compute averages or dates invites errors. A design in which code computes all numbers and the model only phrases them is rarely described.

G4. **Silent handling of missing data.** Systems treat a missing reading as "no data" without saying whether a sensor was switched off, silent or disconnected. This matters once battery-saving switches exist.

G5. **Overstated certainty.** Diagnosis tools commonly present a single label and a percentage. Evidence from the literature [23, 13] argues for possibility-phrased output with visible evidence and an uncertainty caveat.

G6. **Isolated pipelines.** Chat and diagnosis are normally separate products. Sharing a knowledge store lets the assistant later discuss a diagnosis together with the field conditions at that time.

2.6 COMMON FINDINGS FROM LITERATURE

- Retrieval and tool use, not model size alone, are the standard way to bring domain and local knowledge into a general model [5, 6, 21].
- General vision-language models are reliable for coarse health assessment but unreliable for fine-grained identification, so interfaces should constrain them and present results as possibilities [14, 13].
- Model confidence is informative but not calibrated and should be disclosed as such [23, 10, 11].
- Hybrid lexical and dense retrieval is robust and simple to implement [26].
- Hallucination remains a general risk [9], so outputs should be checked by code wherever the answer can be verified (numbers, citation identifiers).

CHAPTER 3

REQUIREMENTS SPECIFICATION

3.1 HARDWARE REQUIREMENTS

3.1.1 Rover Components

Table 3.1 lists the hardware used for the rover and the computing devices around it. Figure 3.1 shows how the parts are connected.

Table 3.1: Hardware components.

Component	Qty	Role
ESP32-CAM (AI-Thinker, OV2640 camera, PSRAM)	1	Wi-Fi, HTTP API, MJPEG live video, high-resolution stills, telemetry push to the backend [31]
Arduino Nano (ATmega328P)	2	Nano 1 reads the sensors, drives the soil-probe servo and the motor driver and talks to the ESP32. Nano 2 is dedicated to the HC-SR04 ultrasonic sensor in the build; see the note below
DHT11 sensor	1	Air temperature and relative humidity (about $\pm 2\,^{\circ}$C and $\pm 5\,\%$ RH)
MQ-5 and MQ-6 gas sensors	1 + 1	Natural gas / methane and LPG / butane, as uncalibrated relative indices
Soil-moisture probe	1	Soil moisture, mounted on a servo-controlled arm and read only when the rover is parked
Servo motor (about 90° travel)	1	Lowers and raises the soil probe
HC-SR04 ultrasonic sensor	1	Obstacle distance for smooth navigation and automatic stop
DC geared motors, 300 RPM	2	Locomotion
L298N dual motor driver	1	Drives the two motors from the battery
Li-ion cells, 3.7 V	4	2S2P pack: about 7.4 V nominal and 8.4 V fully charged
5 V regulator and electronic switches	1 + 2	Supply for the controllers; switches in the MQ-sensor supply lines (see below)
Smartphone (Android)	1	Runs the Flutter application
Laptop or server	1	Runs the backend, database and (optionally) the demonstration data loader
Wi-Fi router, or the rover's own hotspot	1	Network between phone, rover and backend

Note on the controllers. The firmware described in this report reads the ultrasonic sensor on the same Nano as the other sensors. The second Nano of the build is therefore a spare

or a future extension: if the ultrasonic sensor is moved to it, the ESP32 would need a second UART link, while the backend and the application are unaffected because they only see the telemetry.

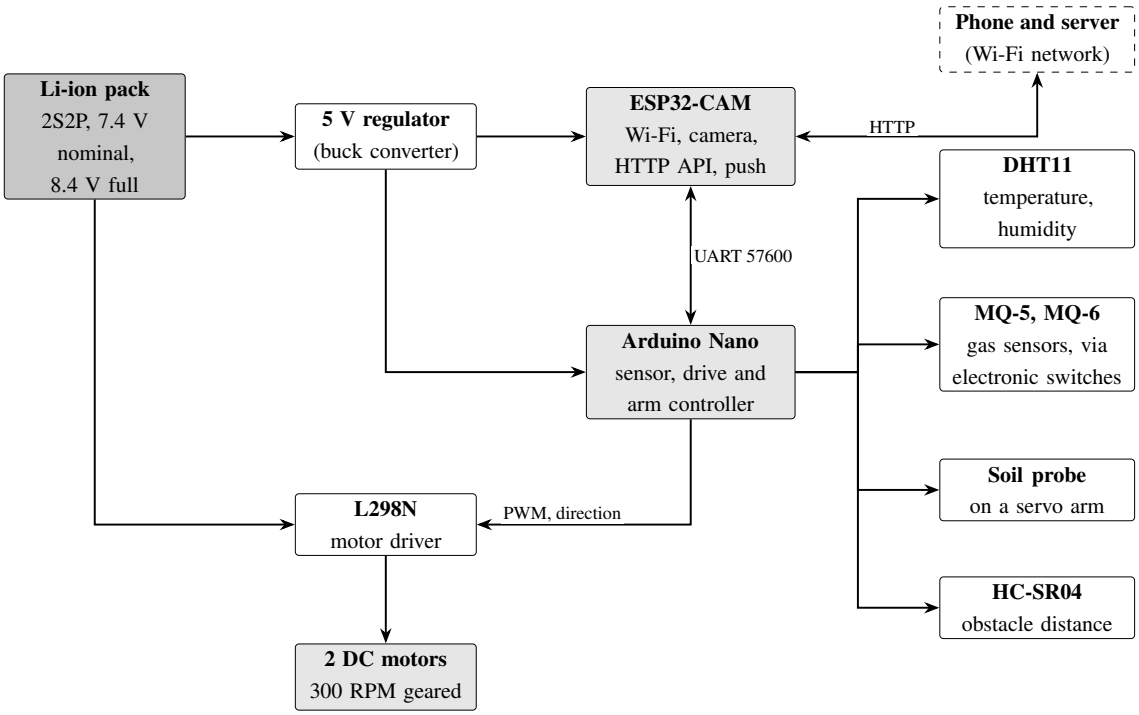


Figure 3.1: Hardware block diagram of the rover.

3.1.2 Power and Safety Constraints

The following constraints, which come from the electrical design, were taken into account:

- The 2S2P pack gives about 7.4 V nominal and 8.4 V when full. It needs suitable cell protection, charging and wiring, and a current rating appropriate for the motors.
- The 5 V output of the L298N board can be used as a test supply but is not guaranteed to stay stable under video streaming load. The exact board must be measured under load, and a dedicated regulator is recommended.
- The two MQ heaters alone can draw about 0.38 A at 5 V, before module circuitry or servo current. A Nano fed from a 9 V battery cannot be assumed to supply the heaters and the servo through its on-board regulator.
- Separate 5 V outputs must not be joined. Interconnected boards need a common ground, and the 5 V Nano transmit line must be level-shifted before it enters an ESP32 pin.

- A software toggle that merely stops reading a sensor does not remove its power consumption. Power is saved only if the supply itself is switched, which requires an electronic switch (relay or logic-level MOSFET module) driven by a controller pin. A sensor must never be powered directly from a microcontroller pin.

3.2 SOFTWARE REQUIREMENTS

Table 3.2: Software and versions used.

Layer	**Software**	**Purpose**
Backend runtime	Python 3.12, FastAPI, Uvicorn [32]	REST and streaming API
Data access	SQLAlchemy 2, psycopg 3, Pydantic 2	ORM, database driver, schema validation
Database	PostgreSQL 16, pgvector [25]	Relational data, vector index (HNSW, cosine) and full-text index (GIN)
Generative AI SDK	google-genai, httpx	Gemini API, and OpenAI-compatible providers [30]
Web search	ddgs (DuckDuckGo), Tavily API	Keyless default, optional keyed search [33]
Images and documents	Pillow, pypdf	Image validation and resizing; PDF text extraction
Mobile application	Flutter 3.35, Dart, http, image_picker, shared_preferences	Android application [34]
Firmware	Arduino core (AVR 1.8.8, ESP32 3.3.12), ArduinoJson 7, DHT sensor library, Servo	Nano and ESP32-CAM sketches
Quality	pytest, flutter_test, arduino-cli	Automated tests and firmware compilation
Delivery	GitHub Actions, Docker Compose	APK build, test runs, backend and database containers

An API key for the Gemini API is required for the default configuration; it is placed in the backend's `.env` file and never in the mobile application. Web search needs no key

by default.

3.3 FUNCTIONAL REQUIREMENTS

Table 3.3: Functional requirements.

ID	Requirement
FR1	The rover shall stream live video and provide a high-resolution still on request.
FR2	The app shall drive the rover with a D-pad or joystick, with an automatic stop if commands cease.
FR3	The rover shall measure temperature, humidity, gas indices and (when parked with the probe down) soil moisture.
FR4	The rover shall push timestamped readings to the backend over Wi-Fi, buffering them while the backend is unreachable.
FR5	Each sensor shall be switchable on or off from the app; the backend shall record every switch change.
FR6	The backend shall validate readings, reject invalid values and store them without duplicates.
FR7	The backend shall aggregate readings into 15-minute windows and day summaries and write them as text chunks with embeddings.
FR8	"Query my farm" shall answer from exact statistics and retrieved chunks, citing every farm-specific statement.
FR9	The assistant shall explain missing data (switched off, no reading, rover not connected) from recorded facts.
FR10	"Open-ended" mode shall answer with cited web search results.
FR11	Chat answers shall be streamed to the app as they are written.
FR12	The app shall accept a photograph from the rover camera, the phone camera or the gallery for diagnosis.
FR13	Diagnosis shall return strict JSON from one vision-language model call (plant, status, evidence, confidence).
FR14	Diagnosis shall run one web research agent and present findings with sources.
FR15	Diagnosis findings shall be written one way into the shared knowledge store; deletion shall remove them.
FR16	The app shall show a Live view of readings arriving, windows closing and chunks being written, embedded and stored.

ID	Requirement
FR17	The backend shall expose a REST API for the above and a small web interface.
FR18	The system shall run without any API key (reduced mode) and report which capabilities are unavailable.

3.4 NON-FUNCTIONAL REQUIREMENTS

Table 3.4: Non-functional requirements.

ID	Quality	Requirement
NFR1	Correctness	Every number in a farm answer shall come from code, and citation identifiers that are not in the retrieved context shall be removed.
NFR2	Honesty	Model confidence shall be shown as a rough, uncalibrated indicator, and diagnoses shall be phrased as possibilities.
NFR3	Independence	Pipeline B shall not read sensor or chat tables, and Pipeline A shall not see image pixels; the only link is the one-way write.
NFR4	Responsiveness	The first streamed text shall appear without waiting for the whole answer; slow operations shall not block an HTTP request.
NFR5	Robustness	Provider errors, rate limits and lost connections shall produce a clear message or a retry, not a crash.
NFR6	Safety	Drive commands shall time out, motion shall stop at obstacles, and no exact pesticide doses shall be given.
NFR7	Security and privacy	Telemetry shall need a device key; an optional API key shall protect the API; deleting a diagnosis shall delete its image and chunks.
NFR8	Portability	The backend shall run on Windows, Linux and Docker, and the AI provider shall be changeable by configuration only.
NFR9	Maintainability	Automated tests shall cover ingestion, retrieval, both pipelines, provider contracts and the app.
NFR10	Battery awareness	Sensors shall be individually switchable, and MQ warm-up shall restart whenever a gas sensor is powered on.

3.5 GENERATIVE AI MODEL REQUIREMENTS

Table 3.5: Model roles and requirements.

Role	Requirement	Default choice
Chat and planning LLM	Strict JSON output against a schema, streaming, low latency, adequate reasoning over short contexts	Gemini 2.5 Flash [4]
Vision-language model	Image input, strict JSON output with evidence and uncertainty fields	Gemini 2.5 Flash [4]
Embedding model	Fixed 768-dimension vectors, useful for retrieval, query and document modes	gemini-embedding-001 [27], L2-normalised; offline hashing fallback
Web search	Titles, URLs and snippets; no key required for the default	DuckDuckGo (ddgs), or Tavily if a key is set
Alternatives	Selected by configuration only, without code changes	OpenAI-compatible providers: Groq, xAI Grok, local Ollama

3.6 DATASET / KNOWLEDGE-BASE REQUIREMENTS

GAIA does not train a model, so it needs no training dataset. It needs a *knowledge base* and, for evaluation, labelled data:

- **Sensor log.** Long-format readings (device, sensor, time, value) with a registry that defines each sensor's unit, valid range and interpretation bands. New sensor keys are accepted without code changes.
- **Knowledge chunks.** Text with a 768-dimension embedding, a source type, a source identifier and optional start and end times. Source types are 15-minute windows, daily summaries, sensor on/off logs, diagnoses, agent findings, farmer notes and documents.
- **Evaluation data.** A deterministic synthetic farm (three days of rover sessions with a planted gas spike, a diagnosis and farmer notes) whose ground truth is computed directly from the raw readings, and a seeded sample of labelled PlantVillage leaf photographs for the vision step [17, 18].

- **Demonstration data.** Four days of synthetic rover sessions, five farmer notes, three farm documents and four freely licensed leaf photographs, loaded through the real API.

CHAPTER 4

SYSTEM DESIGN

4.1 PROPOSED SYSTEM ARCHITECTURE

GAIA is organised in four layers (Figure 4.1): the rover, the Flutter mobile application, the backend services and the data layer. Two outside services are used: the Gemini API (chat, vision and embeddings) and web search.

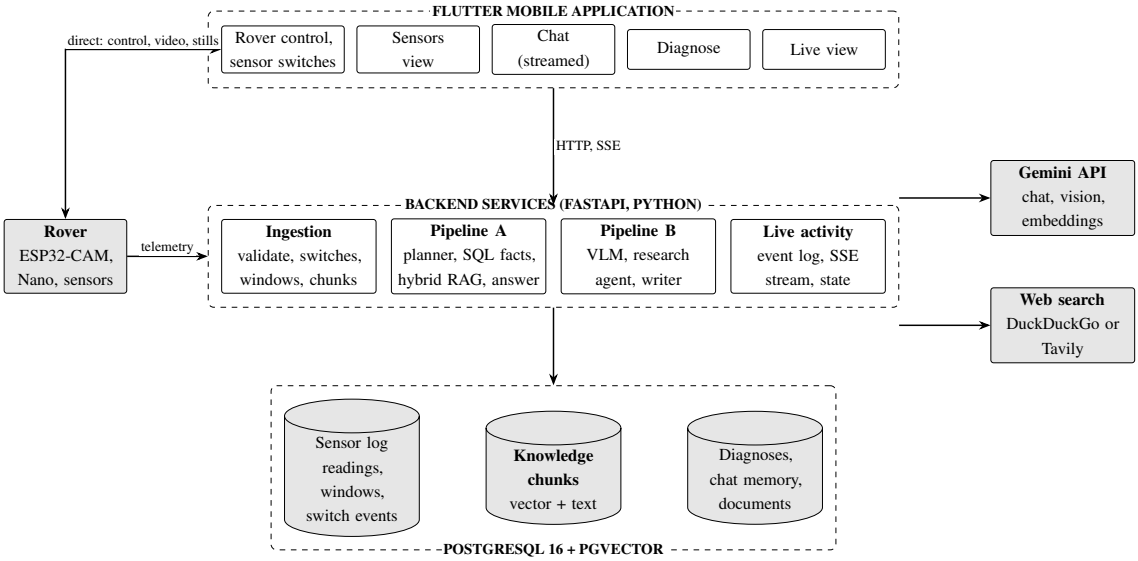


Figure 4.1: Layered architecture of GAIA.

4.1.1 Design Principles

1. **Numbers come from code, words come from the model.** Sensor statistics are computed by SQL and by deterministic templates. The language model receives these facts and phrases them. It is never asked to compute an average or a date.
2. **Two independent pipelines.** Pipeline B never reads sensor data or chat history, and Pipeline A never sees image pixels. The only connection is that Pipeline B *writes* text chunks that Pipeline A may later *read*. There is no control loop. A test records every SQL statement issued during a diagnosis and checks that none touches the sensor or chat tables.

3. **No feedback from chat into the store.** Chat answers are saved as conversation memory only. They are not embedded into the knowledge store, so a model's mistake cannot return later as "farm knowledge".
4. **Rover as a thin layer.** The rover samples, queues and posts. All interpretation happens in the backend.
5. **Degrade gracefully.** Without a model key, chat lists the retrieved facts and the offline hashing embedder keeps retrieval working.
6. **Explain gaps from records.** Which sensors were on, off or silent is recorded by code, so the assistant never has to guess.

4.1.2 Network Topologies

The phone talks *directly* to the rover for control, live video and still capture; none of this passes through the backend. The rover pushes sensor readings to the backend by itself. The phone talks to the backend for sensor views, chat and diagnosis (Figure 4.2). If the phone is connected to the rover's own hotspot, the rover has no route to the backend. The app then offers a *relay*: it reads the rover's latest reading every five seconds and forwards it to the same backend endpoint, queueing up to one hour of readings while the backend is unreachable. The rover's firmware supports three Wi-Fi modes (station with hotspot fallback, hotspot only, or hotspot plus station).

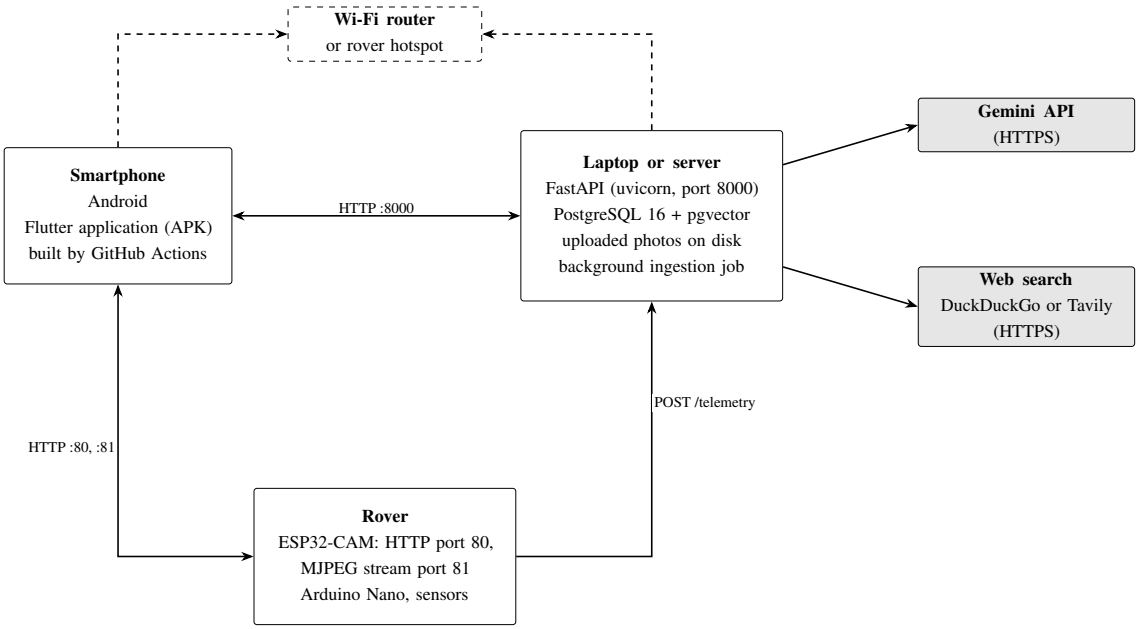


Figure 4.2: Deployment view: devices, ports and protocols.

4.2 SYSTEM BLOCK DIAGRAM

Figure 4.3 follows the data from the sensors to the knowledge store (row A), from a question to a streamed answer (row B) and from a photograph to a stored diagnosis (row C). The knowledge store is the single meeting point.

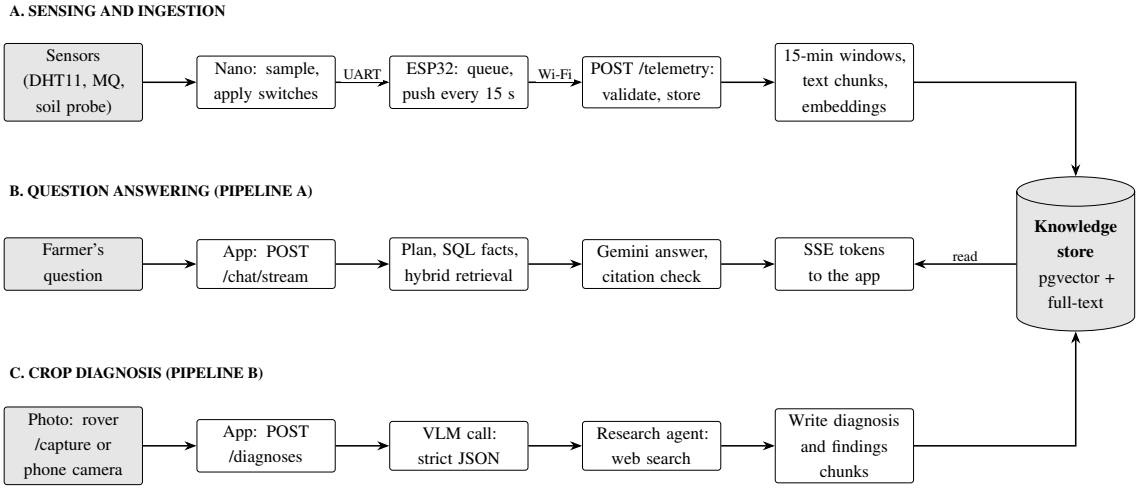


Figure 4.3: System block diagram with the three data paths.

4.2.1 Use Cases

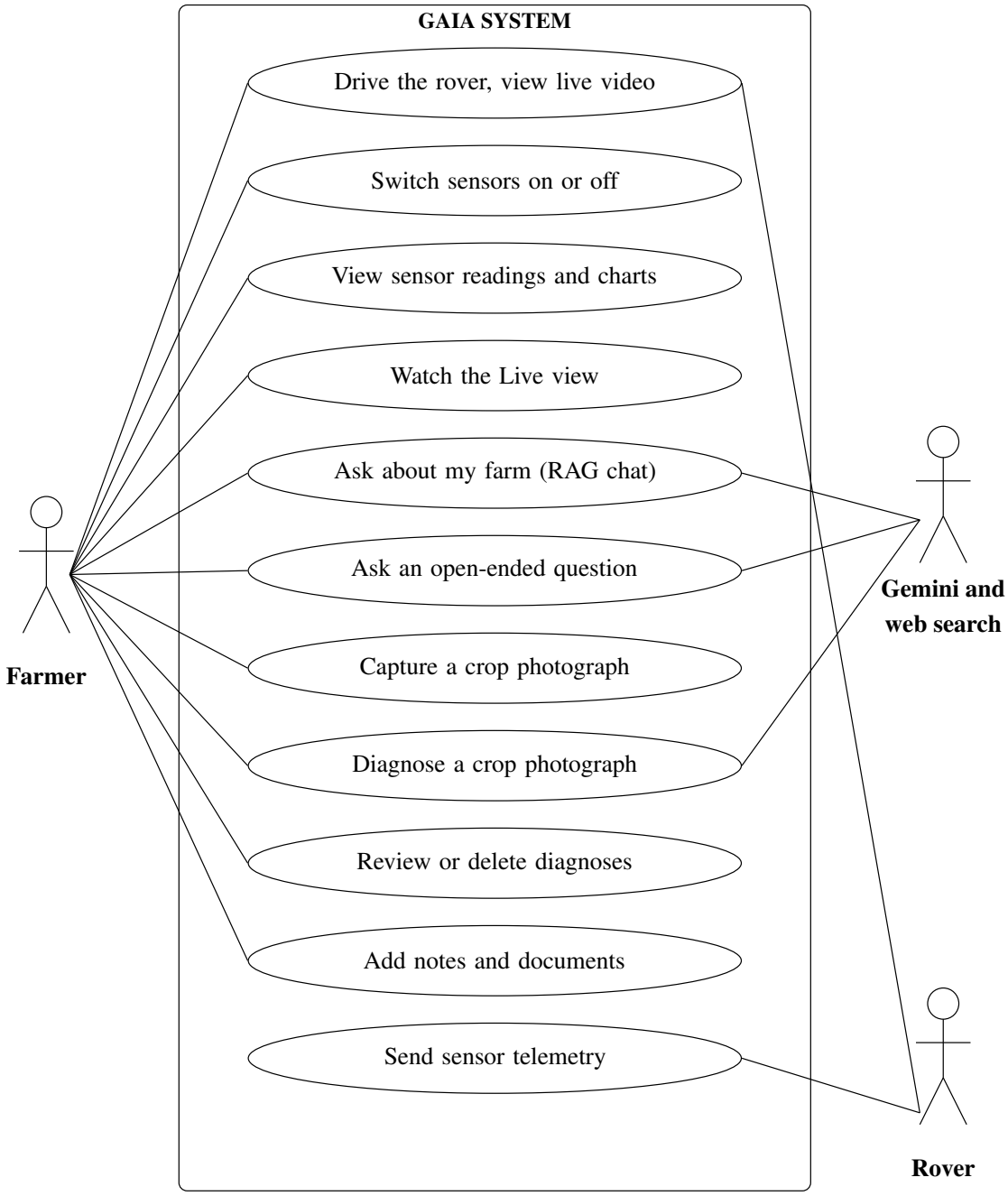


Figure 4.4: Use-case diagram.

4.2.2 Data Model

Figure 4.5 shows the relational schema. Sensor readings are stored in *long format* (one row per sensor per reading), so a new sensor needs no schema change. Chunks are linked logically to their sources by a source type and a source identifier, not by foreign keys, so

one table can hold every kind of chunk. Vectors have 768 dimensions and are indexed with HNSW (cosine); a GIN index on the text supports full-text search.

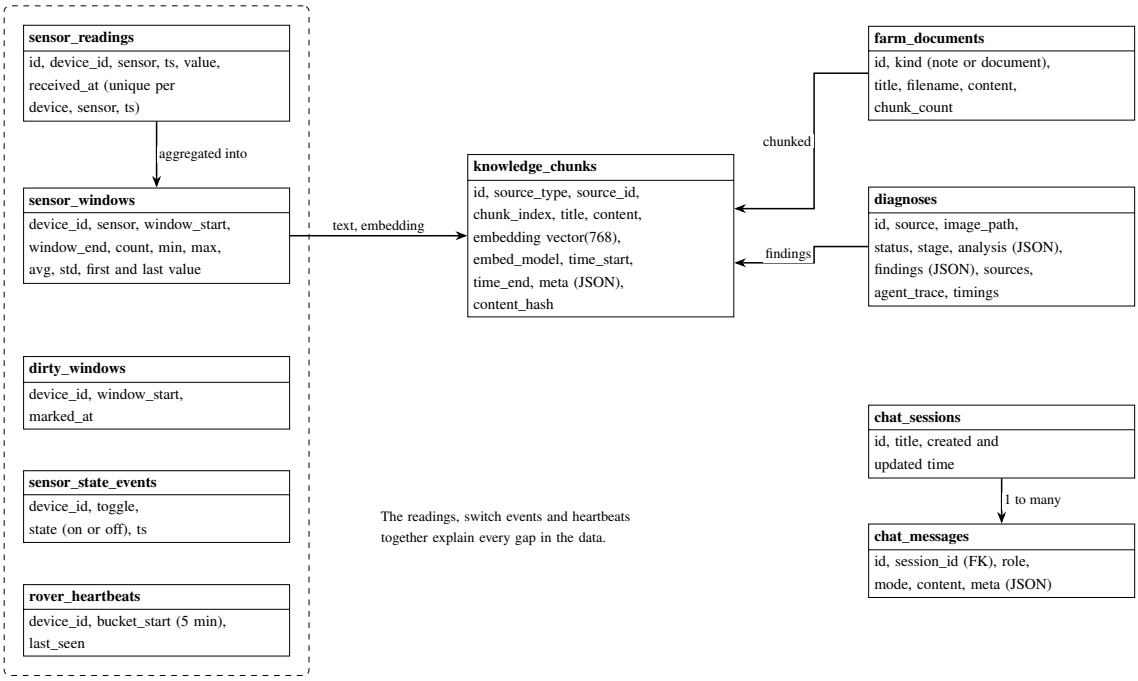


Figure 4.5: Entity–relationship diagram of the database.

4.3 SYSTEM WORKFLOW

4.3.1 Data Lifecycle from Reading to Searchable Chunk

The ESP32 samples the Nano's latest reading every five seconds, keeps up to about 30 minutes of readings in a ring buffer while the backend is unreachable, and posts a batch every 15 seconds. The backend validates and stores each value immediately, so the newest numbers are readable through the API at once. Closed windows are aggregated and turned into text, and the embedding makes them searchable between 90 and 150 seconds after the window ends (a 90-second grace period for late data plus a background job that runs every 60 seconds). Every chat question also forces an aggregation of the still-open window, so *exact statistics* are always current (Figure 4.6).

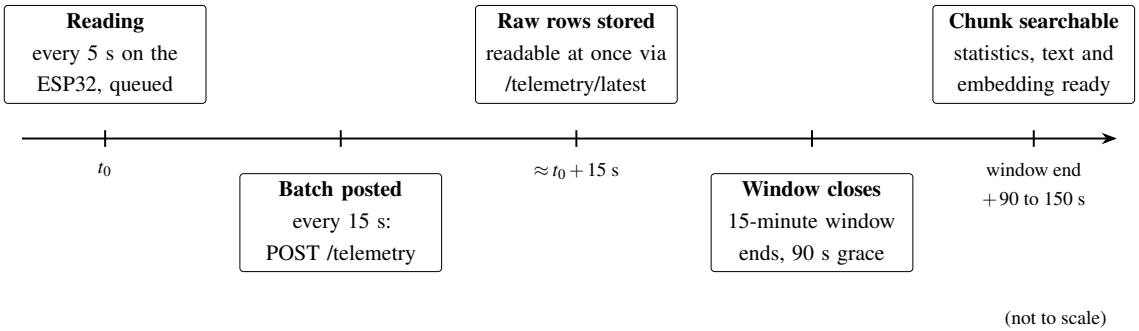


Figure 4.6: Lifecycle of a sensor reading.

Three layers of text are produced from the numbers. All are written by deterministic templates:

1. a **window chunk** for each 15-minute window (statistics per sensor and the interpretation band of its average);
2. a **daily summary** (extremes with their times, a time-of-day profile and threshold events);
3. a **sensor on/off log** for the day (when the rover was connected and which sensors were on or off).

An optional fourth chunk, an AI-written interpretation of the day, can be enabled; it is labelled as interpretation and is written only from the daily summary text.

4.3.2 Pipeline A: Query My Farm

Figure 4.7 shows the steps. The question is first turned into a structured *query plan* by the language model (or by a rule-based planner if no model is available): the intent (current status, statistics, comparison, trend, diagnosis history, farm knowledge, general advice or small talk), the sensors involved, and time *tokens* such as `day:-1` or `daypart:0:morning`. Deterministic code resolves these tokens to exact ranges in the farm's time zone, so the model never computes a date. Exact statistics become numbered *facts* (`[F1]`, `[F2]...`), hybrid retrieval returns numbered *chunks* (`[K1]...`), and the answer-generation call receives only this context. Citation identifiers that are not in the context are removed from the answer, and the response carries flags that say whether the answer used farm data, the knowledge store, or general knowledge.

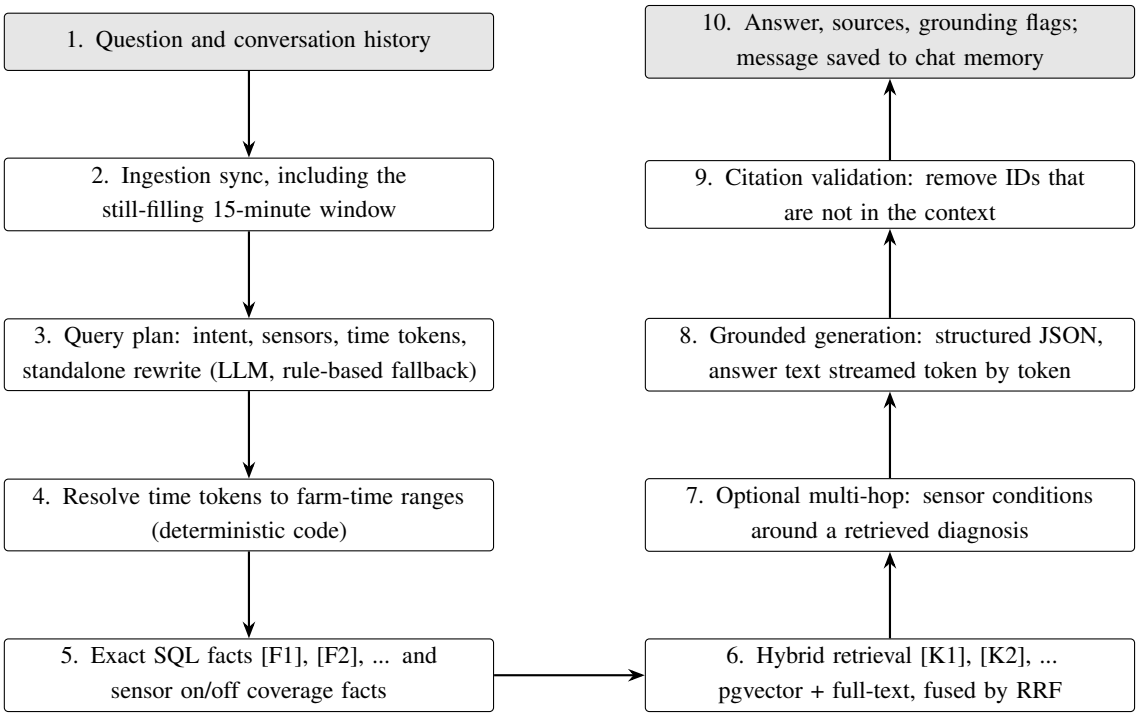


Figure 4.7: Workflow of Pipeline A ("Query my farm").

Open-ended mode. The model plans zero to three web queries; the code runs them in parallel, removes duplicates, ranks institutional sources (universities, government and extension services) first, labels each result's quality and drops forum and social-media results when better sources exist. The answer cites results as `[W1]`, `[W2]`... and the citations are validated in the same way.

4.3.3 Pipeline B: Crop Diagnosis

Figure 4.8 shows the steps. One vision-language call returns strict JSON. If the image is not a plant, the job ends. Otherwise one research agent searches the web: healthy plants lead to cultivation and care information, unhealthy plants to the condition, treatment and prevention, and *uncertain* results are treated like unhealthy ones plus a search for how to tell the suspected condition from its look-alikes. The agent is bounded to two rounds of plan, search and observe. A final call writes the findings with `[W#]` citations. The result is shown to the farmer and then written one way into the knowledge store.

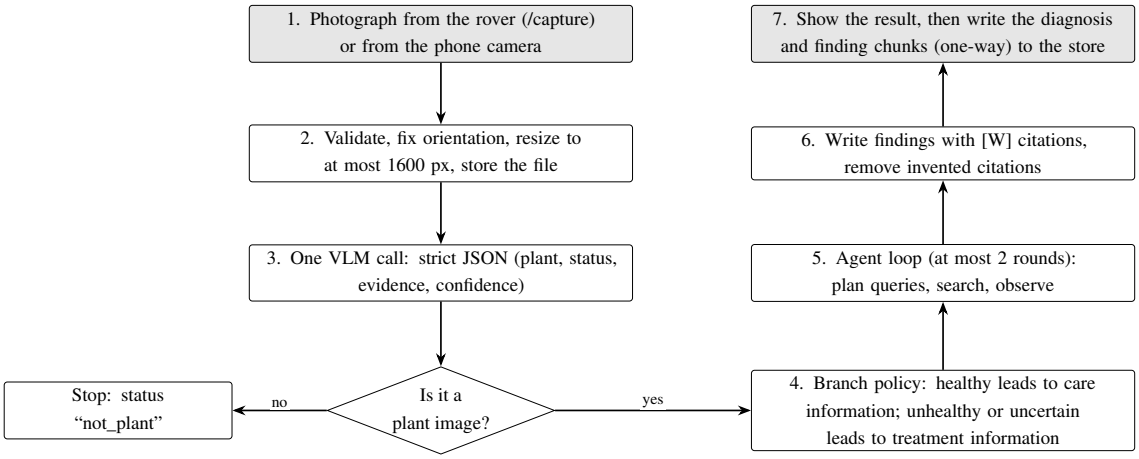


Figure 4.8: Workflow of Pipeline B (crop diagnosis).

A diagnosis runs as a background job so that slow mobile networks never hold a long HTTP request: the upload returns at once and the app polls the job's stage (Figure 4.9).

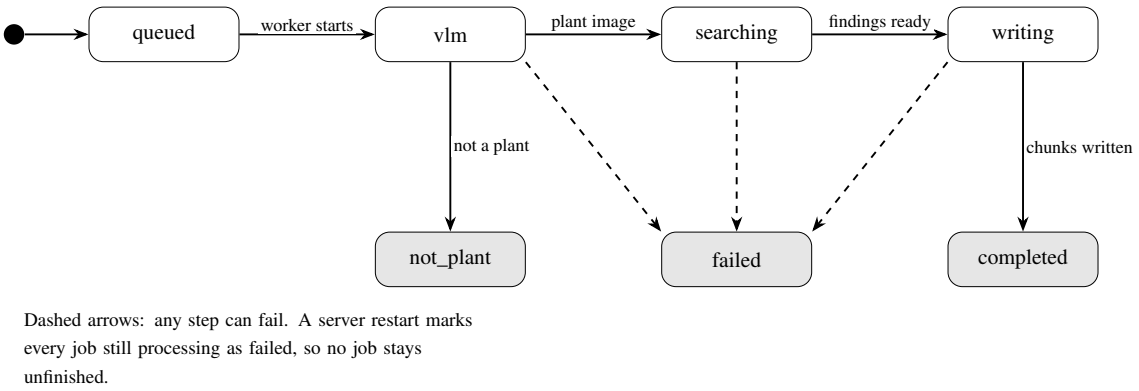


Figure 4.9: State machine of a diagnosis job.

4.3.4 Streaming a Chat Answer

The chat endpoint streams Server-Sent Events [35]: the stage names (syncing, planning, retrieving, generating), the answer text as it is written, and finally the complete answer with validated citations, which replaces the streamed draft. The model still returns one structured JSON object in a single call; the backend decodes the `answer` field of this JSON as it arrives, so the phone shows words immediately without a second model call (Figure 4.10).

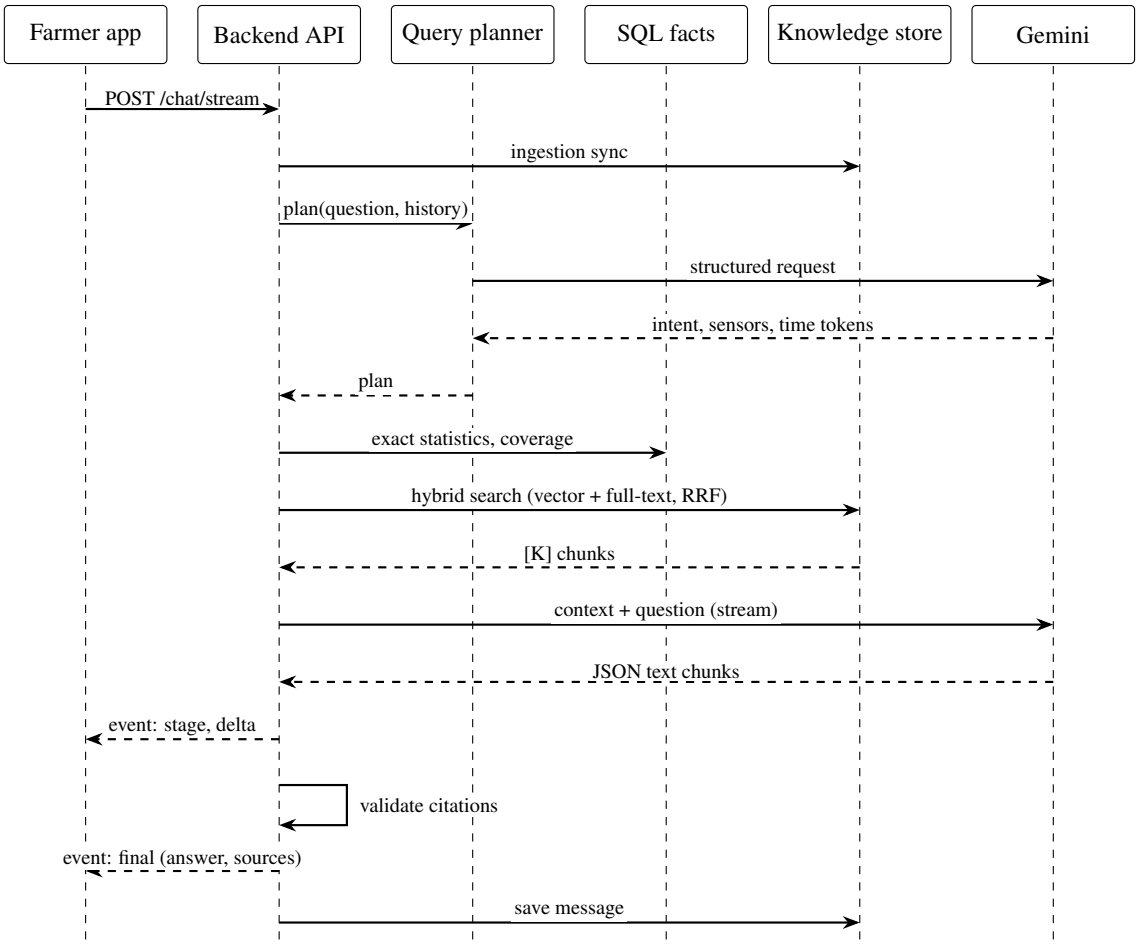


Figure 4.10: Sequence diagram of a streamed "Query my farm" answer.

4.3.5 Sensor Switches and Data Gaps

Each sensor can be switched on or off from the app, which saves battery because the MQ heaters dominate the rover's sensor load. The rover reports its switches with every reading and every push. A push with no readings is a *heartbeat*, which shows that the rover is connected even when every sensor is off (video-only mode). From the switch events, the readings and the heartbeats, every sensor in every five-minute bucket gets one status (Figure 4.11): *recorded*, *off*, *no reading* (switched on but silent: the heater is warming up or the soil probe is not lowered), *offline* (the rover is not connected) or *unknown* (no switch information, for example older firmware).

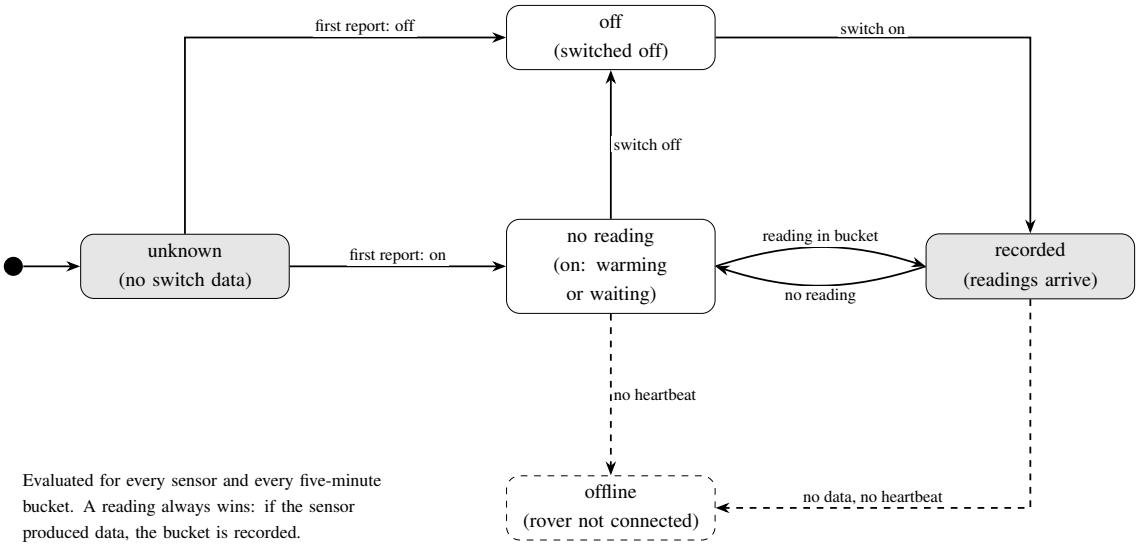


Figure 4.11: Status of a sensor in a five-minute bucket.

These statuses are used in three places. Window chunks and the daily summary state which sensors were missing and why. A dedicated *sensor on/off log* chunk lists the rover's sessions. At question time, a *switches* fact states when each requested sensor was recorded, off or silent, and attaches a tip ("switch MQ5 on in the app") when the sensor is off. The answer prompt instructs the model to quote these periods exactly, never to estimate values for them, and to say which period an average covers.

4.3.6 Live Activity View

Every step publishes a short event into an in-memory log (readings received, window closed, chunk created, chunk embedded, chunk stored, sensor switched, chat and diagnosis stages). The backend exposes this log as a snapshot and as a Server-Sent Events stream, and the app's Live view animates the sensor-to-knowledge-store pipeline from it.

4.4 MATHEMATICAL MODEL

Symbols are defined in the Nomenclature section.

4.4.1 Time Reconstruction

The ESP32 has no real-time clock. A reading may carry a valid network-time stamp; otherwise it carries its uptime u_i, and the batch carries the uptime u^{sent} at sending. The

backend reconstructs

$$\hat{t}_i = t^{\text{recv}} - \frac{u^{\text{sent}} - u_i}{1000} \quad \text{(seconds)}, \qquad u^{\text{sent}} \geq u_i, \tag{4.1}$$

so a reading that was queued during an outage still receives the correct time. A value that is more than five minutes in the future is rejected.

4.4.2 Windows and Statistics

Windows are aligned to the farm's local clock: $W_k = [t_0 + k\Delta,\ t_0 + (k+1)\Delta)$ where t_0 is local midnight and Δ divides 1440 minutes. For sensor s with $n_{s,k}$ readings in W_k, the stored statistics are

$$\mu_{s,k} = \frac{1}{n_{s,k}} \sum_i x_{s,i}, \qquad \sigma_{s,k} = \sqrt{\frac{1}{n_{s,k}} \sum_i \left(x_{s,i} - \mu_{s,k}\right)^2}, \qquad m_{s,k} = \min_i x_{s,i}, \quad M_{s,k} = \max_i x_{s,i}. \tag{4.2}$$

Statistics over a longer period R (a day, a week) are combined without returning to the raw rows. The mean is count-weighted:

$$\mu_s(R) = \frac{\sum_{k \in R} n_{s,k}\, \mu_{s,k}}{\sum_{k \in R} n_{s,k}}, \qquad \min_R = \min_{k \in R} m_{s,k}, \qquad \max_R = \max_{k \in R} M_{s,k}. \tag{4.3}$$

4.4.3 Hybrid Retrieval

Embeddings are L2-normalised, so the cosine similarity is a dot product:

$$\cos(\mathbf{q}, \mathbf{d}) = \mathbf{q} \cdot \mathbf{d}, \qquad \text{pgvector distance} = 1 - \cos(\mathbf{q}, \mathbf{d}). \tag{4.4}$$

A vector ranking and a full-text ranking are fused with Reciprocal Rank Fusion [26]:

$$\text{RRF}(d) = \sum_{r \in \{\text{vector}, \text{text}\}} \frac{1}{k + \text{rank}_r(d)}, \qquad k = 60, \tag{4.5}$$

where a term is omitted if d does not appear in ranking r. Chunks are returned in decreasing order of $\text{RRF}(d)$.

4.4.4 Sensor Status

Let Rec_s be the set of buckets in which sensor s produced readings, Seen the set of buckets in which the rover was connected (a batch arrived or any reading exists), and $\text{sw}_s(b) \in$

$\{\text{on}, \text{off}, \varnothing\}$ the latest switch state at the end of bucket b.

$$\text{st}_s(b) = \begin{cases} \text{recorded} & b \in \text{Rec}_s \\ \text{offline} & b \notin \text{Rec}_s,\ b \notin \text{Seen} \\ \text{off} & b \in \text{Seen},\ \text{sw}_s(b) = \text{off} \\ \text{no reading} & b \in \text{Seen},\ \text{sw}_s(b) = \text{on} \\ \text{unknown} & \text{otherwise.} \end{cases} \tag{4.6}$$

Readiness of a gas sensor and validity of a soil reading are

$$\text{ready}_s(t) = \text{on}_s(t) \wedge \big(t - t_{\text{on},s} \geq T_w\big), \qquad \text{valid}_{\text{soil}}(t) = \text{on}_{\text{soil}} \wedge \text{arm} = \text{deployed} \wedge \neg\,\text{moving}, \tag{4.7}$$

with $T_w = 60\,\text{s}$ and $t_{\text{on},s}$ the time the sensor was last switched on.

4.4.5 Calibration Metrics

For N diagnoses with self-reported confidence p_i and correctness $y_i \in \{0,1\}$, grouped into bins B_j, the expected calibration error [23] and the Brier score [36] are

$$\text{ECE} = \sum_j \frac{|B_j|}{N} \big|\,\text{acc}(B_j) - \text{conf}(B_j)\,\big|, \qquad \text{BS} = \frac{1}{N}\sum_{i=1}^{N} (p_i - y_i)^2. \tag{4.8}$$

4.4.6 Rover Conversions and Energy

The battery voltage is read through a divider and the soil probe is mapped linearly between a dry and a wet calibration value:

$$V_{\text{bat}} = \frac{\text{ADC}}{1023} V_{\text{ref}} \frac{R_1 + R_2}{R_2}, \qquad \text{soil}\,[\%] = \text{clamp}\left(100\,\frac{\text{raw} - \text{raw}_{\text{dry}}}{\text{raw}_{\text{wet}} - \text{raw}_{\text{dry}}}, 0, 100\right), \tag{4.9}$$

with $V_{\text{ref}} = 5\,\text{V}$, $R_1 = 10\,\text{k}\Omega$, $R_2 = 4.7\,\text{k}\Omega$, $\text{raw}_{\text{dry}} = 620$ and $\text{raw}_{\text{wet}} = 280$ (to be recalibrated per probe). The ultrasonic distance in centimetres is the echo time in microseconds divided by 58, and joystick input is mixed as left $=$ throttle $+$ steer, right $=$ throttle $-$ steer.

The energy saved by switching a sensor off for time t_{off} is $\Delta E = P_h t_{\text{off}}$ when its supply is switched. For the two MQ heaters, $P_h \approx 5\,\text{V} \times 0.38\,\text{A} = 1.9\,\text{W}$. As an illustration, a pack of two parallel strings of cells with an assumed 2000 mAh per cell (replace with the datasheet value) stores about $2 \times 2.0\,\text{Ah} \times 7.4\,\text{V} = 29.6\,\text{Wh}$, so keeping the gas sensors off for one hour saves about $1.9/29.6 \approx 6\%$ of the pack, ignoring regulator losses.

CHAPTER 5

PROJECT IMPLEMENTATION

5.1 OVERVIEW OF PROJECT MODULES

The repository is organised into two main folders, `frontend` (the Flutter application) and `backend` (all services and pipeline logic), plus the rover firmware, demonstration data and documentation. Table 5.1 maps each module to its location.

Table 5.1: Project modules and their locations.

Module	Location	Responsibility
Rover firmware	`firmware/`	Nano controller, ESP32-CAM server, UART protocol
Ingestion	`backend/gaia/ingestion`	Validation, switches, windows, text chunks
Knowledge store	`backend/gaia/knowledge`	Chunk upserts, hybrid retrieval
Pipeline A	`backend/gaia/pipeline_a`	Planner, SQL facts, farm RAG, open chat, streaming
Pipeline B	`backend/gaia/pipeline_b`	VLM, research agent, diagnosis jobs
Providers	`backend/gaia/providers`	LLM and VLM clients, embeddings, web search
API and live view	`backend/gaia/api`, `core/activity.py`	REST and SSE endpoints, activity log
Web interface	`backend/gaia/web`	Browser version of the app and a retrieval inspector
Tools and evaluation	`backend/scripts`	Rover simulator, bring-up tools, evaluations
Mobile application	`frontend/lib`	Screens, services, models
Demonstration data	`demo_data/`	Sample history, notes, documents, photos and loader
Delivery	`.github/workflows`, Docker files	APK build, tests, containers

5.2 DATA COLLECTION AND PREPROCESSING

5.2.1 From the Sensor to the Backend

The Nano samples the sensors and sends a checksummed text frame to the ESP32 twice a second. The ESP32 converts the latest frame into JSON every five seconds, queues it, and posts a batch every 15 seconds (or immediately after a sensor is switched). An example batch that the ESP32 sends when it has no clock is shown below. Each reading carries only its uptime; the backend reconstructs the time with Equation 4.1.

```
POST /api/v1/telemetry          X-Device-Key: <key>
{"device_id": "rover-01", "sent_uptime_ms": 130000,
 "sensor_switches": {"dht11": true, "soil_moisture": true, "mq5": true, "mq6":
     true},
 "readings": [
   {"uptime_ms": 120000,
    "values": {"temperature": 27.5, "humidity": 61.0, "soil_moisture": 43.5,
               "mq5": 210, "mq6": 190, "distance_cm": 91, "battery_v": 7.85},
    "state": {"moving": false, "arm_deployed": true, "soil_valid": true}} ]}
```

5.2.2 Validation

For each value the backend normalises the sensor name (for example `temp` to `temperature`), rejects non-numeric and out-of-range values using the sensor registry, drops soil moisture unless the probe state says the arm is deployed and the rover is still, ignores duplicates (the same device, sensor and time), and rejects timestamps more than five minutes in the future. The response reports how many values were accepted, rejected and duplicated, with a short list of warnings.

5.2.3 Aggregation and Text Generation

Windows that received new readings are marked "dirty". A background job (every 60 seconds) and every chat question run a *sync*: closed windows are re-aggregated (Equation 4.2), written as text and embedded, and the dirty mark is removed only if no new reading arrived in the meantime, so late data from the ring buffer is never lost. A window chunk looks like this (a real example from the demonstration run):

```
Field sensor readings from rover-01 on Saturday 17 October 2026, 16:00-16:15
(afternoon, farm time).
- Air temperature: average 32.8 C (min 32.1, max 33.5, 15 readings) - hot:
  heat stress possible, especially with low soil moisture.
- Relative humidity: average 49% (min 45%, max 52%, 15 readings) - moderate.
```

```
- Soil moisture: average 39.2% (min 38.4%, max 40.0%, 10 readings) - adequate.
- MQ5 gas index: average 210 ADC (min 180, max 240, 15 readings) - baseline.
```

If a sensor has no statistics in a window, the chunk says why, for example "MQ5 gas index: not recorded in this window — sensor switched off". Chunks whose text is unchanged are not embedded again (a content hash is compared), and each chunk records the embedding model that produced its vector, so a change of model never mixes vector spaces (a re-index endpoint re-embeds all chunks).

5.2.4 Image Preprocessing

Uploaded photographs are validated with Pillow (non-images and decompression bombs are rejected), rotated according to their EXIF orientation, resized to at most 1600 pixels and saved as JPEG. The farmer's optional note is stored as metadata but is *not* shown to the model, which keeps Pipeline B image-only.

5.3 GENERATIVE AI MODEL SELECTION

- **Chat and planning: Gemini 2.5 Flash.** It supports a response schema (structured output), streaming, and is fast and inexpensive enough for several calls per question. It is configurable (`LLM_MODEL`); newer models can be selected without code changes.
- **Vision: Gemini 2.5 Flash.** The same model accepts images, so one API key serves both roles. The vision model is configurable separately (`VLM_MODEL`).
- **Embeddings: gemini-embedding-001 at 768 dimensions**, L2-normalised [27]. Queries and documents use the corresponding task types. An offline hashing embedder is the fallback so that retrieval works without a key.
- **Alternatives.** Any OpenAI-compatible service (Groq with Llama models, xAI Grok, local Ollama) can replace the chat or the vision model through configuration. The pipelines use only two provider operations, `complete` (free text) and `structured` (validated JSON, plus a streaming variant), so provider changes are configuration, not code. The local-model path was tested end to end with Gemma served by Ollama.
- **Web search.** DuckDuckGo through the `ddgs` package needs no key and is the default; Tavily is used when its key is set.

5.4 MODEL CONFIGURATION / FINE-TUNING

No model is fine-tuned. The reasons are those of Table 2.2: the knowledge that matters (the farm's own measurements) changes every few seconds and cannot be absorbed by training, while retrieval gives fresh, citable context. Behaviour is controlled by prompts, response schemas and a few settings (Table 5.2).

Table 5.2: Main backend settings (`backend/.env`) and defaults.

Setting	Default	Meaning
`GEMINI_API_KEY`	none	Enables chat, vision and embeddings
`WINDOW_MINUTES`	15	Aggregation window (must divide 1440)
`INGEST_GRACE_SECONDS`	90	Wait for late data before a window is closed
`INGEST_INTERVAL_SECONDS`	60	Background indexing period
`RAG_TOP_K`	6	Chunks retrieved per question
`CHAT_HISTORY_TURNS`	6	Conversation turns sent to the model
`AGENT_MAX_ROUNDS`	2	Research agent rounds
`AGENT_QUERIES_PER_ROUND`	3	Search queries per round
`MAX_UPLOAD_MB`, `VLM_MAX_IMAGE_PX`	12, 1600	Photo limits
`DEVICE_KEY`, `API_KEY`	dev key, none	Rover telemetry key; optional app key
`FARM_TIMEZONE`, `FARM_REGION`	Asia/Kolkata, India	Time resolution; region added to web searches

Prompting rules for answers. The farm-answer prompt requires that farm-specific statements come from the context and carry a citation; that `[F#]` numbers be quoted and not recomputed; that diagnoses be called possible or suspected; that missing data be stated plainly with the available data mentioned; that general advice appear only in a final paragraph beginning with "General guidance:"; that MQ values never be presented as ppm; that every "unusual reading" fact be mentioned even if averages look normal; and that switched-off sensors and disconnected periods be reported exactly without estimating values. The model returns a structured object (`answer`, `cited`,

used_general_knowledge, insufficient_data, follow_ups).

5.5 PROJECT MODULES

5.5.1 Rover Firmware

Arduino Nano. The sketch (about 12.6 KB of flash, 608 bytes of RAM) runs motors with a soft ramp (so the gearboxes are not shocked), a 700 ms command watchdog, an automatic forward stop at 20 cm (adjustable), and a soil-arm state machine (retracted, deploying, settling, deployed, retracting) that lifts the probe before any movement and reports soil only when the arm is deployed, settled and the rover is still. Sensors are read at their own periods (DHT11 every 2 s, ultrasonic every 60 ms). Each sensor has an enable flag; switching an MQ sensor on restarts its 60-second warm-up, and an optional pin drives an electronic switch in its supply line.

UART protocol. Frames have the form `$BODY*CS` at 57600 baud, where CS is the XOR of the body characters in two hex digits; frames with a bad checksum are ignored. Commands from the ESP32 are `D` (drive), `S` (stop), `A` (arm), `C` (configure), `W` (sensor switch) and `P` (ping); the Nano replies `K` (acknowledged) or `E` (rejected). Telemetry is sent at 2 Hz. Example (MQ-2 not fitted):

```
$T,812,274,610,-1,208,191,435,84,3,7860,56,15,15,15*75
   seq, temp x10, hum x10, mq2, mq5, mq6, soil x10, distance, arm,
   battery mV, flags, installed mask, enabled mask, warm mask
```

ESP32-CAM. It joins the configured Wi-Fi (falling back to its own hotspot), serves the control API on port 80 and the MJPEG stream on port 81 from a separate server so that streaming never blocks control, and pushes telemetry. Table 5.3 lists the API. The stream is VGA at about 14 frames per second; stills are UXGA (1600×1200). The drive watchdog stops the motors if no command arrives for 500 ms, and failed pushes back off exponentially up to two minutes.

Table 5.3: HTTP API of the ESP32-CAM.

Method	Path	Purpose
GET	`:81/stream`	MJPEG live video
GET	`/capture`	One high-resolution JPEG (optional flash)
GET	`/snapshot`	One JPEG at stream resolution
GET	`/api/status`	Wi-Fi, Nano link, drive, arm, battery, push state
GET	`/api/telemetry`	Latest reading in the backend's shape
POST	`/api/drive`	`left/right`, `throttle/steer` or `dir`, repeated every 100–150 ms
POST	`/api/stop`	Stop at once
POST	`/api/arm`	Deploy or retract the soil probe
GET, POST	`/api/sensors`	Read or set the per-sensor switches (`{"all": false}` is video-only mode)
POST	`/api/config`	Obstacle distance and maximum PWM

5.5.2 Backend: Ingestion and Knowledge Store

The ingestion service (Algorithm 1) and the sync job (Algorithm 2) implement the lifecycle of Figure 4.6. The knowledge store offers one write operation, *upsert a source*, which replaces all chunks of a logical source (for example one window or one diagnosis) and skips re-embedding when the content hash is unchanged. Every chunk written publishes three activity events (created, embedded, stored), which is how the Live view shows all kinds of chunks.

Algorithm 1 Ingest a telemetry batch

Require: batch with device id, readings, optional switches, receive time t^{recv}

record a heartbeat in the five-minute bucket of t^{recv}
for all reading r in the batch **do**
    $\hat{t} \leftarrow$ NTP time if valid, else Equation 4.1, else t^{recv} for a lone reading
    if $\hat{t}$ is missing or more than 5 minutes in the future **then**
        reject the values; **continue**
    end if
    for all (name, value) in r **do**
        normalise the name; reject non-finite or out-of-range values
        if name is soil moisture **and not** soil valid **then**
            reject
        end if
        add (sensor, $\hat{t}$, value) to the row set
    end for
    if r carries switches **then**
        add ($\hat{t}$, switches) to the switch timeline
    end if
end for
add (batch switches, t^{recv}) to the timeline
for each change of a switch state in time order **do** insert a switch event
insert rows ignoring duplicates; mark the affected windows dirty
return counts of accepted, rejected and duplicate values

Algorithm 2 Sync: aggregate, write and embed windows

Require: current time t, flag include_open

cutoff $\leftarrow t - \Delta -$ grace
for all dirty window w in time order **do**
    closed $\leftarrow$ (w.start $\leq$ cutoff)
    if not closed **and not** include_open **then**
        continue
    end if
    aggregate the window (Equation 4.2)
    if not closed **then**
        continue ▷ statistics are current, indexing waits
    end if
    reasons $\leftarrow$ why sensors are missing in w (switch and heartbeat records)
    text $\leftarrow$ window chunk from the statistics and reasons
    upsert the chunk (embed unless the hash is unchanged)
    clear the dirty mark only if it was not re-marked meanwhile
end for
for all day touched by a closed window **do**
    rebuild the daily summary and the sensor on/off log; upsert both
end for

5.5.3 Pipeline A: Planner, Facts and Retrieval

Planner. The planner returns a structured query plan. A rule-based planner with the same output is used when no model is available or the call fails, and deterministic backstops correct typical model mistakes (for example, a question about a named past period is never "current status").

Facts. The fact builder computes, from SQL over the window table, the latest readings (with their age and a warning if stale), the statistics of each requested sensor and period, comparisons and their differences, time-of-day profiles, "unusual reading" facts for short excursions that an average would hide, linked conditions around a diagnosis (plus or minus one hour), log coverage, and the sensor on/off history described in Section 4.3. If a requested sensor has no data, a "no data" fact is followed by the reason.

Retrieval. Algorithm 3 shows the hybrid search. Source types are filtered by intent (only diagnoses for diagnosis questions) and sensor chunks by time overlap, while diagnoses and notes are searched across all time.

Algorithm 3 Hybrid retrieval with Reciprocal Rank Fusion

Require: query text, k_{top}, optional source types, optional time range

q $\leftarrow$ normalised embedding of the query (query mode)
$V \leftarrow$ top candidates by cosine distance (HNSW index) under the filters
$T \leftarrow$ top candidates by PostgreSQL full-text rank under the same filters
for all chunk d in $V \cup T$ **do**
    $\text{score}(d) \leftarrow \sum_{r \in \{V,T\}: d \in r} 1/(60 + \text{rank}_r(d))$
end for
return the k_{top} chunks with the highest score, with their vector and lexical ranks

5.5.4 Pipeline A: Grounded Generation and Streaming

Algorithm 4 gives the answer procedure. The streaming decoder (Algorithm 5) extracts the text of the `answer` property from the partial JSON the model is writing. It handles escape sequences and Unicode surrogate pairs split across network chunks. At the end the whole JSON is validated, with one non-streaming regeneration if it is malformed, and the final answer replaces the streamed draft.

Algorithm 4 Answer a farm question

Require: question, history, time t

sync ingestion including the open window
plan $\leftarrow$ query plan (LLM or rules); ranges $\leftarrow$ resolve time tokens in farm time
facts $\leftarrow$ SQL facts for plan.sensors and ranges, with coverage facts
chunks $\leftarrow$ hybrid retrieval of plan.search_query if plan.search_knowledge
if plan.link_conditions **then**
    add facts for ± 1 h around each retrieved diagnosis
end if
context $\leftarrow$ facts $\cup$ chunks, numbered `[F#]` and `[K#]`
out $\leftarrow$ structured LLM answer from context and question (streamed if requested)
remove citations whose identifier is not in the context
return answer, cited sources, grounding flags, follow-up questions

Algorithm 5 Incremental decoding of the answer field

buffer ← empty; state ← seeking
for all text chunk *c* produced by the model **do**
    append *c* to buffer
    if state is seeking **and** buffer contains `"answer": "` **then**
        state ← inside; position ← end of match
    end if
    while state is inside **and** a complete character is available at position **do**
        decode one character (handle escapes and surrogate pairs); emit it to the app
        if the character is an unescaped quote **then**
            state ← done
        end if
    end while
end for
parse and validate the whole JSON (one non-streaming retry if invalid)

5.5.5 Pipeline B: Vision Analysis and Research Agent

Vision step. One call with a fixed system prompt returns JSON with the following fields: whether the image is a plant, the plant and its part, `health_status` (healthy, unhealthy or uncertain), a diagnosis phrased as a possibility, issue categories (disease, nutrient deficiency, pest damage, fungal, bacterial or viral infection, environmental stress, physical damage, abnormal coloration or morphology), *evidence* (what is directly visible), *assumptions* (what is inferred), a differential list, severity, a rough confidence and the image quality. After the call the code re-enforces invariants (a non-plant image is always "uncertain") and attaches a fixed caveat that the confidence is not calibrated.

Research agent. Algorithm 6 shows the loop. The agent is "one agent": a bounded loop in code with the model making the decisions, not a tool-calling framework, so it works identically with every provider.

Algorithm 6 Research agent for a diagnosis

Require: VLM JSON a, search tool, $R_{\max} = 2$ rounds, $Q_{\max} = 3$ queries per round

results ← empty; asked ← empty
for round = 1 **to** $R_{\max}$ **do**
    step ← LLM plan from a, the branch policy, asked queries and results so far
    queries ← new queries from step (first $Q_{\max}$); **if** round = 1 and none, use fallback queries
    if step is done (after round 1) **or** no queries **then**
        break
    end if
    run the queries in parallel; keep new URLs only, at most 12 sources; rank institutional sources first
end for
findings ← LLM writer from a and the numbered sources `[W#]`
remove invalid and invented citations and placeholder rows; **return** findings, sources, trace

Presenting and writing. The result page shows the plant, the status, the evidence, the confidence with its caveat, recapture tips for poor images, the findings and the sources. A "diagnosis" chunk and one or more "agent finding" chunks are written with provenance (URLs, domains, model identifiers, whether the answer was web-grounded). Deleting a diagnosis removes the stored image and its chunks.

5.5.6 Sensor Switches and Coverage Engine

The rover reports its switches; the backend stores only changes in `sensor_state_events` and one row per five-minute bucket in `rover_heartbeats`. The coverage engine (Algorithm 7) evaluates Equation 4.6 for a time range. Consecutive "on" buckets (recorded or no reading) are merged for display, because a probe that only measures while parked alternates between the two statuses all the time, which is normal and not a gap. Descriptions list time spans for ranges up to 36 hours and totals for longer ones. Rover channels that cannot be switched (distance, battery) are not covered.

Algorithm 7 Coverage of one sensor switch over a range

Require: device, switch s, range $[a, b)$ clipped to the current time

1: load the switch events before b, the buckets with heartbeats, and the buckets with readings (any sensor, and sensor s)
2: **for all** five-minute bucket β in $[a, b)$ **do**
3: assign $\mathrm{st}_s(\beta)$ by Equation 4.6
4: **end for**
5: merge adjacent buckets with the same status into spans; merge adjacent recorded and no-reading spans
6: **return** spans, and the text: "readings recorded ...; switched off ...; rover not connected for ..."

5.5.7 Live Activity View

An in-memory log keeps the last 400 events with increasing identifiers. `GET /live/state` returns the progress of the open window, counters, the current switch state and recent events; `GET /live/stream` streams new events as Server-Sent Events, with a heartbeat comment every 15 seconds and resume from an identifier. The Live view shows six boxes (rover sensors, server receives, collecting a window, text chunk, embedding, knowledge store) that light up when an event for them arrives and whose connectors animate while data flows, a second lane for chat and diagnosis stages, and a feed of recent events.

5.5.8 Flutter Mobile Application

The application has six tabs: **Overview**, **Rover** (live MJPEG video, D-pad or joystick with a 120 ms heartbeat, soil-probe control, per-sensor switches with a "video only" button, and capture), **Diagnose** (camera, gallery or rover photo, live job stages, results with sources), **Gaia AI** (both chat modes, streamed answers, citations that open the source, grounding badges, follow-up suggestions, a persistent session), **Sensors** (latest values, charts, an "off" badge for switched-off sensors) and **Live**. Services (rover link, backend client, repositories) are created once and provided through an inherited widget. The MJPEG decoder cuts frames at the JPEG start and end markers, which works with any camera boundary format. A small Server-Sent Events parser handles chunk boundaries, and the chat falls back to the non-streaming endpoint if the server does not support streaming. Settings hold the backend and rover addresses, an optional API key and the relay switch.

5.5.9 Web Interface, Simulator and Tools

The backend also serves a browser version of the application (overview, rover, diagnosis, chat, sensors) and a knowledge-store inspector that shows the vector, lexical and fused scores of each chunk. A *rover simulator* implements the same HTTP API as the firmware (including the sensor switches and heater warm-up), so the whole system can be tested without hardware. `nano_console` talks the UART protocol to a Nano on USB with automated checks, and `rover_check` walks the assembled rover's API. A demonstration loader sends sample data through the real API.

5.5.10 Delivery

A GitHub Actions workflow builds the Android APK from the `frontend` folder on demand and on changes to it. A second workflow runs the backend tests (including an embedded PostgreSQL), an offline evaluation, and the application's analysis and tests. Docker Compose starts the backend with a `pgvector` PostgreSQL image.

5.6 TOOLS AND TECHNOLOGIES USED

Table 5.4: Tools and technologies.

Area	**Tools**
Hardware and firmware	ESP32-CAM, Arduino Nano, Arduino C++, `arduino-cli`, ArduinoJson, DHT sensor library
Backend	Python 3.12, FastAPI, Uvicorn, SQLAlchemy 2, Pydantic 2, httpx, Jinja2
Database	PostgreSQL 16, pgvector (HNSW, cosine), full-text search (GIN), embedded server through `pgserver` for development
Generative AI	Gemini API (`google-genai`), OpenAI-compatible APIs (Groq, Grok, Ollama), DuckDuckGo (`ddgs`), Tavily
Mobile application	Flutter and Dart, `http`, `image_picker`, `shared_preferences`, `url_launcher`
Testing	pytest, `flutter_test`, fake provider servers, in-process fake Nano, rover simulator
Delivery	GitHub Actions, Docker Compose
Documentation	LaTeX (this report), Markdown project documents

5.7 ALGORITHM DETAILS

The algorithms of this chapter are collected here with their cost, using n for the readings in a window, w for the windows touched by a sync, m for the chunks in the store, and c for the number of model calls.

Table 5.5: Algorithm summary.

Algorithm	**Cost**	**Notes**
Ingest a batch (Alg. 1)	$O(r)$ for r values	One bulk insert; duplicates ignored in the database
Sync (Alg. 2)	$O(wn)$ plus embedding	One embedding call per changed chunk
Hybrid retrieval (Alg. 3)	$O(\log m)$ vector search	HNSW index; full-text via GIN index
Answer a question (Alg. 4)	2 model calls	Plan and answer; plus embedding of the query
Answer decoding (Alg. 5)	linear in text length	Constant work per character
Research agent (Alg. 6)	up to 2 plan calls, 1 writer call	Plus 1 vision call; bounded cost
Coverage (Alg. 7)	$O(b)$ for b buckets	Five-minute resolution

CHAPTER 6

SOFTWARE TESTING

6.1 TEST CASES

6.1.1 Test Strategy

Testing combines four layers. (i) *Automated unit and integration tests* run the real backend code against a lightweight database and, for the retrieval layer, against a real PostgreSQL server with pgvector. (ii) *Provider contract tests* run the actual Gemini client and the OpenAI-compatible client against small local servers that imitate the providers, so the exact requests (schema, system prompt, inline image, embedding size) and the parsing of responses are verified without network access or keys. (iii) *Application tests* cover parsing, repositories and widgets, including pages run against a local server that speaks the real endpoints. (iv) *Hardware bring-up tools* test the firmware and the assembled rover. Scripted model doubles replace the language models in the pipeline tests, so the code around the models is verified deterministically. Table 6.1 lists the number of automated tests; all of them pass.

Table 6.1: Automated tests by area (all passing).

Test module	**Tests**	**Area**
`test_pipeline_a`	19	Planner, facts, citations, offline mode, gaps
`test_streaming`	14	Answer decoder, SSE endpoint, provider streams
`test_switches`	9	Sensor switches, coverage, chunks, prompt
`test_api`	8	REST API, security, web pages
`test_provider_contracts`	8	Gemini and OpenAI-compatible clients
`test_pipeline_b`	7	Diagnosis end to end, privacy, failures
`test_rover_tools`	6	UART protocol, simulator, check tools
`test_ingestion`, `test_timeutil`, `test_evaluate_vlm`	5 + 5 + 5	Ingestion, time resolution, vision metrics
`test_live`	4	Activity events and streams
`test_demo_data`, `test_postgres`	3 + 3	Demo loader; real PostgreSQL and pgvector
Backend total	**96**	
Application (`frontend/test`)	22	Parsers, repositories, widgets

6.1.2 Selected Test Cases

Table 6.2: Selected functional test cases.

ID	**Test (module)**	**Expected result**	**Result**
TC1	Telemetry without a device key (`test_api`)	HTTP 401; with the key, 200 and the latest values appear	Pass
TC2	Validation, soil gating, uptime timestamps (`test_ingestion`)	Invalid values rejected; soil dropped unless the probe is deployed; time rebuilt from uptime	Pass
TC3	Duplicate batch (`test_ingestion`)	Second post stores nothing new	Pass

ID	Test (module)	Expected result	Result
TC4	Late data re-aggregation (`test_ingestion`)	Open window stays dirty; late readings are re-aggregated, not lost	Pass
TC5	Unchanged window (`test_ingestion`)	Not embedded again (content hash)	Pass
TC6	Hybrid search on PostgreSQL (`test_postgres`)	HNSW and full-text indexes exist; fused ranks returned	Pass
TC7	Exact facts and citation validation (`test_pipeline_a`)	Averages and deltas in the prompt; invented `[F42]` removed	Pass
TC8	Missing data (`test_pipeline_a`)	Answer says no readings and sets the insufficient-data flag	Pass
TC9	Unusual readings (`test_pipeline_a`)	A short gas spike hidden by the average becomes its own fact	Pass
TC10	Gas question with MQ5 and MQ6 only (`test_pipeline_a`)	No facts about an MQ2 sensor that is not fitted	Pass
TC11	Diagnosis end to end (`test_pipeline_b`)	Stages advance; diagnosis and findings chunks written; sources listed	Pass
TC12	Independence (`test_pipeline_b`)	No SQL statement touches sensor or chat tables during a diagnosis	Pass
TC13	Pipeline A reads Pipeline B (`test_pipeline_b`)	Chat finds a stored diagnosis and its finding	Pass
TC14	Not a plant (`test_pipeline_b`)	Agent and knowledge write skipped	Pass
TC15	Fake citations and placeholders (`test_pipeline_b`)	Invented citations and “N/A” rows removed from findings	Pass
TC16	Streamed chat (`test_streaming`)	Stages, many text deltas, validated final answer; message saved	Pass
TC17	Provider stream (`test_streaming`)	Real Gemini and OpenAI-compatible clients parse SSE chunks	Pass
TC18	Provider failure (`test_streaming`)	Stream ends with an error event, not a crash	Pass
TC19	Switch events (`test_switches`)	Event stored only when the state changes; heartbeat accepted	Pass
TC20	Coverage statuses (`test_switches`)	“recorded 06:00–07:00; switched off 07:00–08:00” and offline time	Pass
TC21	Gaps in the prompt (`test_switches`)	Model receives the on/off history and the tip	Pass
TC22	Legacy telemetry (`test_switches`)	Without switches the behaviour is unchanged	Pass

ID	Test (module)	Expected result	Result
TC23	UART frames (`test_rover_tools`)	Checksums valid; old and new frame formats decode	Pass
TC24	Rover check against the simulator	Status, camera, stills, arm, sensor switches, watchdog pass	Pass
TC25	Live page against a local server (app)	Pipeline boxes, chips, feed and switch state update as events arrive	Pass

6.1.3 Hardware and Firmware Verification

- **Compilation.** Both sketches compile without warnings in the project sources with `arduino-cli` (all warnings enabled). The Nano sketch uses 12,644 bytes of flash (41 %) and 608 bytes of RAM (29 %). The ESP32-CAM sketch uses 1.23 MB of flash (39 %) and 86 KB of RAM (26 %).
- **Nano bring-up tool.** `nano_console` checks the link, ping, the telemetry rate (about 2 frames per second), the checksum error count, DHT11 values, each gas sensor, the ultrasonic sensor, battery, configuration commands, the soil-arm sequence and its interlock, the drive pulses and the failsafe, and it switches the first gas sensor off and on again while watching the frames. It is tested against an in-process fake Nano that mirrors the firmware's command handling.
- **Rover check tool.** `rover_check` walks the assembled rover's HTTP API (status, sensors, switches, snapshot and still, video stream, arm, drive and watchdog) and prints PASS, WARN or FAIL with a wiring hint. It is tested against the simulator.
- **On the physical rover.**
  Record here the results of running both tools on your assembled rover, and the photographs of the rover (see Chapter 7).

6.2 GENERATIVE AI OUTPUT TESTING

6.2.1 What Is Tested and Why

Generative output cannot be tested only by comparing strings, so GAIA tests the properties that matter for a farmer:

1. **Numerical correctness.** Each expected number is computed *directly from the raw readings* (not from the pipeline's own tables) and searched for in the answer.

2. **Grounding.** The answer must cite the right *type* of evidence (an exact statistic for a numeric question, a diagnosis chunk for a diagnosis question, a farmer note for a farm-knowledge question).
3. **Hallucinated citations.** Citation identifiers that do not exist in the context are removed, including grouped forms such as `[W10, W12]`.
4. **Honest gaps.** When data is missing the answer must say so, and when a sensor was switched off it must give the periods from the recorded switches.
5. **Structured output validity.** Output that does not match the schema is repaired once; models that cannot follow a response schema fall back to a prompt-level schema.
6. **Safety and tone.** Diagnoses are possibilities; confidence carries a fixed caveat; no exact pesticide doses; MQ values are never ppm.

6.2.2 Evaluation Set for Pipeline A

A deterministic synthetic farm is generated (three days of rover sessions, a planted gas spike, one diagnosis with findings, and farmer notes). Ten representative questions are scored (Table 6.3). *Planner accuracy* is the fraction of questions for which the intent, sensors and time tokens match the expected plan. *Numeric recall* is the fraction of expected numbers present in the answer. *Citation accuracy* is the fraction of answers that cite the expected evidence types. *Keyword accuracy* checks required words (for example “blight” or “cattle shed”).

Table 6.3: The ten evaluation questions (intent and expected evidence).

No.	Question	Intent	Expected evidence
1	What was the average soil moisture today?	statistics	exact statistic
2	How humid was my field this morning?	statistics	exact statistic
3	Was the soil moisture lower than yesterday?	comparison	comparison fact
4	What was the highest temperature yesterday?	statistics	exact statistic
5	Any unusual gas readings in the last 3 days?	statistics	statistic or excursion
6	What crop problems have been identified in my field?	diagnosis history	diagnosis chunk
7	What treatment did the system find for the tomato problem?	diagnosis history	finding chunk
8	What environmental conditions were recorded when this issue was detected?	diagnosis history	linked conditions
9	Where is the biogas unit?	farm knowledge	farmer note
10	How did the temperature change through the day today?	trend	profile or daily summary

6.2.3 Findings from the First Local-Model Run

When this evaluation was first run with a small local model (Gemma through Ollama), planner accuracy was 90 %, numeric recall 83 %, and citation accuracy 80 %. The failures led to three changes (two in the pipeline and one in the benchmark), after which planner accuracy, numeric recall, citation accuracy and keyword accuracy all reached 100 %:

1. **Excursions hidden by averages.** A short methane spike was invisible in a normal average. Such excursions now become separate "unusual reading" facts, and the prompt requires every one of them to be mentioned.
2. **Planner backstops.** Deterministic rules now correct typical mistakes: a named past period is never "current status", and diagnosis and knowledge questions always search the store and link sensor conditions when asked.
3. **Benchmark fairness.** For trend questions two evidence types are equally valid (a profile fact or the daily summary), so the benchmark accepts either.

6.2.4 Vision Step Evaluation Design

A separate script evaluates the vision step on a seeded sample of 60 labelled PlantVillage leaf photographs (ten classes of tomato, potato, maize and bell pepper: four healthy and six diseased) [17]. It reports plant identification, health-status accuracy ("uncertain" counts as not correct and is reported separately), the missed-disease rate (diseased leaves called healthy, the costly error), the false-alarm rate, whether the labelled disease is named in the top diagnosis or among the look-alikes, the calibration of the self-reported confidence (Equation 4.8 and a reliability table), and the latency. PlantVillage shows single leaves on plain backgrounds, so these numbers are an optimistic upper bound for photographs taken by the rover. The metric code is covered by unit tests; the run with the Gemini model is reported in Chapter 7 where the result is available.

6.3 PERFORMANCE EVALUATION

6.3.1 Test Environment

Measurements were made on a laptop with an Intel Core i5-8365U (1.6 GHz) and 15.9 GB of memory, running Windows 11, Python 3.12, and the embedded PostgreSQL 16 with pgvector. The backend was exercised in process, with the offline embedder and no language model, so these figures measure the system's own code and database, not the network or the model provider. The synthetic data covered 14 days of rover sessions of 6 hours per day sampled every 5 seconds: 60,480 readings and 302,400 values.

6.3.2 Measured Results

Table 6.4: Measured backend performance.

Measurement	**Result**
Ingestion throughput (batches of 60 readings, 1008 requests)	4,978 values per second; 60.8 s in total
Ingestion latency per batch	median 55.2 ms; 95th percentile 90.9 ms
Indexing 336 closed windows and 14 days (aggregate, text, embed, store)	24.4 s in total, 72.5 ms per window
Chat answer, offline (plan, SQL facts, retrieval, extractive answer)	median 288 ms; 95th percentile 624 ms (50 requests)
Hybrid search in a store of 364 chunks	median 202 ms; 95th percentile 255 ms (30 requests)
`GET /telemetry/latest` over 302,400 stored values	median 166 ms
Offline evaluation, ten questions (synthetic farm)	median 11.5 ms per answer
Local language model (Gemma through Ollama), ten questions	median 18.9 s per answer
Local pipeline B run (vision, agent with live web search, write)	44–86 s on a laptop GPU

6.3.3 Discussion

- At about 5,000 values per second, a rover that produces roughly 5 values every 5 seconds (one value per second) is served with a very large margin; the ingestion path is not a bottleneck.
- Indexing a 15-minute window takes about 73 ms with the offline embedder. With the Gemini embedding model the cost is dominated by the network call to the embedding service, one per changed chunk, so a window closes and becomes searchable within seconds of the background job running.
- Latencies of the offline chat and of the search include the in-process request overhead and the embedded PostgreSQL started on the same laptop. With a language model, answer time is dominated by the model calls (two per question), which is why the answer is streamed: the first words reach the phone while the rest is still being written.

- The latest-values query takes a median of 166 ms at 300,000 stored values; it is fast enough for the app's 15-second refresh and would be a candidate for a materialised view if the history grew by orders of magnitude.
- On the microcontrollers the footprint leaves ample room: 41 % of the Nano's flash and 29 % of its RAM, and 39 % and 26 % on the ESP32-CAM.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 PROJECT OUTCOMES

The project produced a working end-to-end system, which is summarised below.

- **Rover and firmware.** Two sketches (Nano and ESP32-CAM) that compile without warnings, a documented HTTP API and UART protocol, per-sensor power switches with heater warm-up handling, and bring-up tools that check the wiring and the API.
- **Backend.** A validated, timestamped ingestion path; deterministic window, daily and on/off-log chunks; a knowledge store with hybrid retrieval; Pipeline A (planner, exact facts, citation-validated answers, streaming); Pipeline B (vision call, bounded research agent, one-way write); a live activity stream; a browser interface; a rover simulator; and demonstration data with a loader.
- **Mobile application.** Six screens (overview, rover control with sensor switches, diagnosis, streamed chat, sensors, Live view) built into an installable Android package by a GitHub Actions workflow.
- **Quality evidence.** 96 backend and 22 application automated tests, provider contract tests against local fake services, a ten-question evaluation with ground truth computed from raw data, and measured performance figures.
- **Responsible-AI features.** Uncalibrated-confidence disclosure, possibility-phrased diagnoses, evidence separated from assumptions, removal of invented citations, explicit gap reporting, deletion of images and chunks, and chemical-safety rules.

7.2 RESULT ANALYSIS

7.2.1 Pipeline A Evaluation

Table 7.1 reports the ten-question evaluation (Section 6.2) in two configurations that need no cloud key: the offline mode (rule-based planner, extractive answers, hashing embedder) and a local language model (Gemma served by Ollama, hashing embedder). The

raw reports are stored with the project (`docs/evaluation-offline.json` and `docs/evaluation-local-llm.json`).

Table 7.1: Pipeline A evaluation on the synthetic farm (10 questions).

Measure	Offline	Local LLM, first run	Local LLM, final
Planner accuracy	100 %	90 %	100 %
Numeric recall	100 %	83 %	100 %
Citation accuracy (expected evidence type)	100 %	80 %	100 %
Keyword accuracy	100 %	100 %	100 %
Answers carrying citations	100 %	–	100 %
Median latency per answer	11.5 ms	13.5 s	18.9 s

Interpretation. The offline result shows that the deterministic part of the system, which computes the facts and retrieves the chunks, is correct on all ten questions. The first local-model run shows the value of the evaluation: it exposed four failures that were traced to missing excursion facts and a planner that mislabelled past periods, and the fixes made the system more robust for every model. These are single runs on a benchmark of ten questions, so they demonstrate correct behaviour on representative questions; they are not a general accuracy claim. A larger question set and a run with the cloud model are the natural next steps.

Add a third column here after running the RAG evaluation script with your Gemini key (see the project README).

7.2.2 Sensor Gaps

The sensor-switch feature was verified by tests that build a day with the gas sensors on, then off, then the rover disconnected, and then a stretch of video-only mode. The tests check the recorded events, the status of each five-minute bucket, the text of the window, daily and on/off-log chunks, the facts, and the prompt that reaches the model. In a live run with the rover simulator, switching MQ5 and MQ6 off through the backend relay reached the rover, the rover's next push updated the backend within seconds, readings stopped, and the Live view showed each switch as an event.

7.3 GENERATED OUTPUT ANALYSIS

7.3.1 A Window Chunk

The text that is embedded for a 15-minute window is deterministic. This is a real chunk from the demonstration data (Section 5.2 shows the format): it contains the numbers with the interpretation band of each average, and nothing that was invented.

7.3.2 An Answer about a Switched-off Sensor

The following facts are the real output of the backend (offline mode, so the facts are listed directly) for the question "What were the gas readings this morning?" in the demonstration data, where the gas sensors had been switched off:

```
- No mq5 gas index (lpg / natural gas / methane) readings were recorded this
  morning (Sat 17 Oct 2026, 05:00–12:00). [F1]
- No mq6 gas index (lpg / butane / propane) readings were recorded this
  morning (Sat 17 Oct 2026, 05:00–12:00). [F2]
- Sensor on/off history this morning (Sat 17 Oct 2026, 05:00–12:00) (farm time)
    :
  MQ5: switched off 06:00–09:20; rover not connected for 3 h 40 min of the
  period. MQ6: switched off 06:00–09:20; rover not connected for 3 h 40 min
  of the period. [F3]
```

With a language model configured, these facts form the context of the answer, and the prompt makes the model state the periods and the tip to switch the sensors on, without estimating any value.

7.3.3 A Diagnosis with a Local Model

A complete Pipeline B run was made with local models: the rover simulator served a Creative-Commons photograph of a tomato leaf with early-blight lesions, a Gemma model acted as vision-language model, one agent ran live DuckDuckGo searches, the findings were written to the store, and a later chat question recalled them. The vision step reported *unhealthy – possible fungal or bacterial leaf spot* with a self-reported confidence of 0.7. It did not name the crop or the specific disease, which is typical of a small model and is exactly why confidence is shown as uncalibrated and the result as a possibility. The agent cited institutional sources first (a National Institutes of Health archive, Vikaspedia and Tamil Nadu Agricultural University). The run completed in 44 to 86 seconds on a laptop with a GPU.

7.4 PERFORMANCE METRICS

Table 7.2 repeats the headline figures of Section 6.3 together with the firmware footprint.

Table 7.2: Headline performance and resource figures.

Metric	**Value**
Ingestion throughput	4,978 values per second
Indexing time per 15-minute window (offline embedder)	72.5 ms
Offline chat answer, median (95th percentile)	288 ms (624 ms)
Hybrid search, median (95th percentile)	202 ms (255 ms)
Searchable after a window closes	90 to 150 seconds
Newest reading visible in the API	immediately after the batch is posted
Nano flash and RAM use	41 % and 29 %
ESP32-CAM flash and RAM use	39 % and 26 %
Automated tests	96 backend and 22 application, all passing

7.5 ACCURACY / RELEVANCE / QUALITY EVALUATION

7.5.1 Pipeline A

On the ten-question evaluation, the final configurations reached 100 % planner accuracy, numeric recall, citation accuracy and keyword accuracy (Table 7.1). Relevance is supported by design: numeric questions are answered from exact statistics and not from similarity search, sensor chunks are filtered by time overlap, and notes and diagnoses are retrieved across all time.

7.5.2 Pipeline B

The vision evaluation script is implemented and unit-tested, and Table 7.3 is reserved for the run with the Gemini model. The literature gives the expectation: general vision-language models handle the coarse healthy-or-diseased decision well but identify the pathogen less reliably [13, 14], so the design reports a possibility with evidence and a

list of look-alikes and treats “uncertain” as a result in its own right, with advice to retake the photograph.

Table 7.3: Vision step on 60 PlantVillage photographs (to be completed).

Measure	**Gemini 2.5 Flash**
Plant identification	
Health-status accuracy (strict, “uncertain” counted as wrong)	
Missed-disease rate (diseased called healthy)	
False-alarm rate (healthy called unhealthy)	
Disease named in the diagnosis (top-1) / among look-alikes (top-k)	
Expected calibration error / Brier score of the confidence	
Median latency per image	

Run the following command from the `backend` folder to fill the table. The sleep option keeps the run under the free tier’s request limit.

```
python -m scripts.evaluate_vlm --sleep 6 --out ../docs/evaluation-vlm-gemini.
    json
```

Because PlantVillage photographs show single leaves on plain backgrounds, the numbers are an upper bound for photographs taken in the field.

7.6 VALIDATION OF RESULTS

- **Independent ground truth.** The expected numbers of the evaluation are computed from the raw readings by separate code, not from the aggregates that the pipeline uses.
- **Two databases.** The same ingestion and retrieval tests run on a lightweight database and on a real PostgreSQL server with pgvector, which checks that the portable code and the production code agree.
- **Contract tests against fake providers.** The real Gemini and OpenAI-compatible clients are run against local servers that record what is sent, so the requests and the parsing are verified without keys.

- **Live end-to-end runs.** The backend was started on the embedded PostgreSQL, the loader and the rover simulator posted data over HTTP, and chat, streaming, the live stream and the sensor switches were exercised with real requests.
- **Independence test.** A test records every SQL statement during a diagnosis and verifies that none touches the sensor or chat tables.

7.7 COMPARISON WITH EXISTING APPROACHES

Table 7.4: Feature comparison (a dash means not claimed).

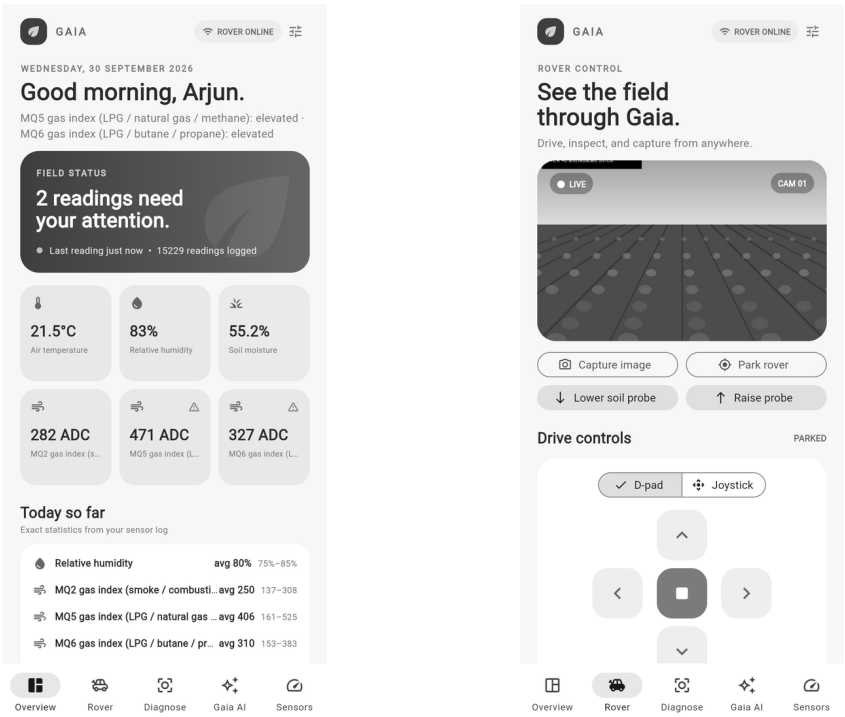
Feature	Photo app [12]	Leaf CNN [17]	General chatbot	Agri RAG [22]	GAIA
Image diagnosis	Yes	Yes (label)	Some	–	Yes
Natural-language Q&A	–	No	Yes	Yes	Yes
Web retrieval with sources	–	No	No	Yes	Yes
Uses the farm's own sensor log	No	No	No	No	Yes
Exact numbers computed by code	–	–	No	–	Yes
Validated citations	–	–	No	–	Yes
Explains missing data	No	No	No	No	Yes
Confidence disclosed as uncalibrated	–	–	–	–	Yes
Proactive field inspection (rover)	No	No	No	No	Yes
Works without a model key (reduced)	–	–	No	–	Yes

GAIA's contribution is not a better leaf classifier. It is the combination of the farm's own measurements, exact computation, validated citations and honest handling of gaps and uncertainty, which none of the compared approaches reports.

7.8 SCREENSHOTS OF THE IMPLEMENTED SYSTEM

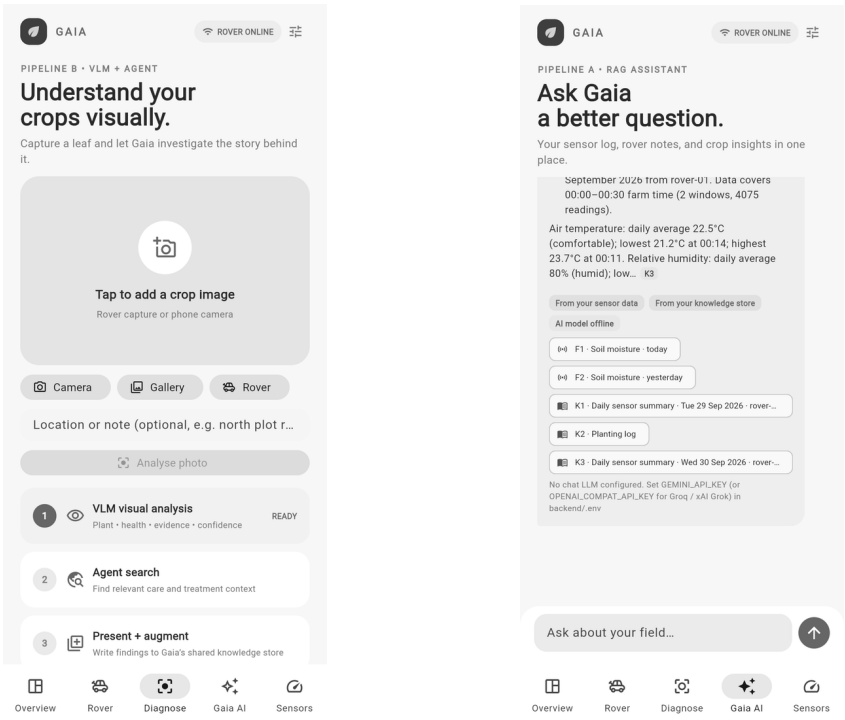
The screenshots below were captured from the running system with the rover simulator, converted to grey scale for printing.

They predate the sensor switches, the streamed chat and the Live view: replace them with newer screenshots (names are given in the empty frames) and add the photographs of the physical rover.



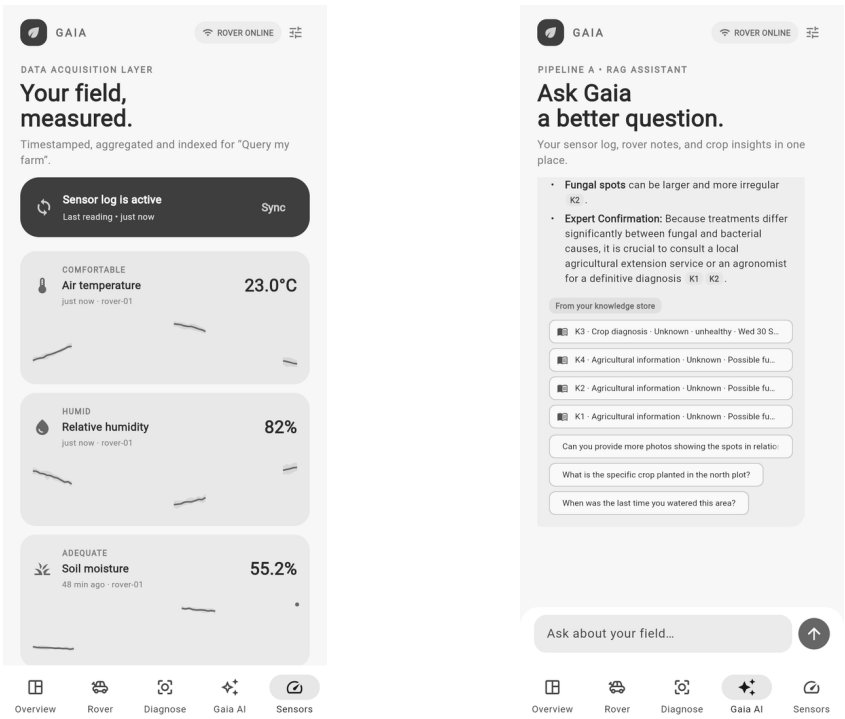
(a) Overview (b) Rover control

Figure 7.1: Mobile application: overview and rover control.



(a) Diagnose (b) Gaia AI chat

Figure 7.2: Mobile application: diagnosis and chat.



(a) Sensors (b) Chat answer from a local model

Figure 7.3: Mobile application: sensor values and a cited chat answer.

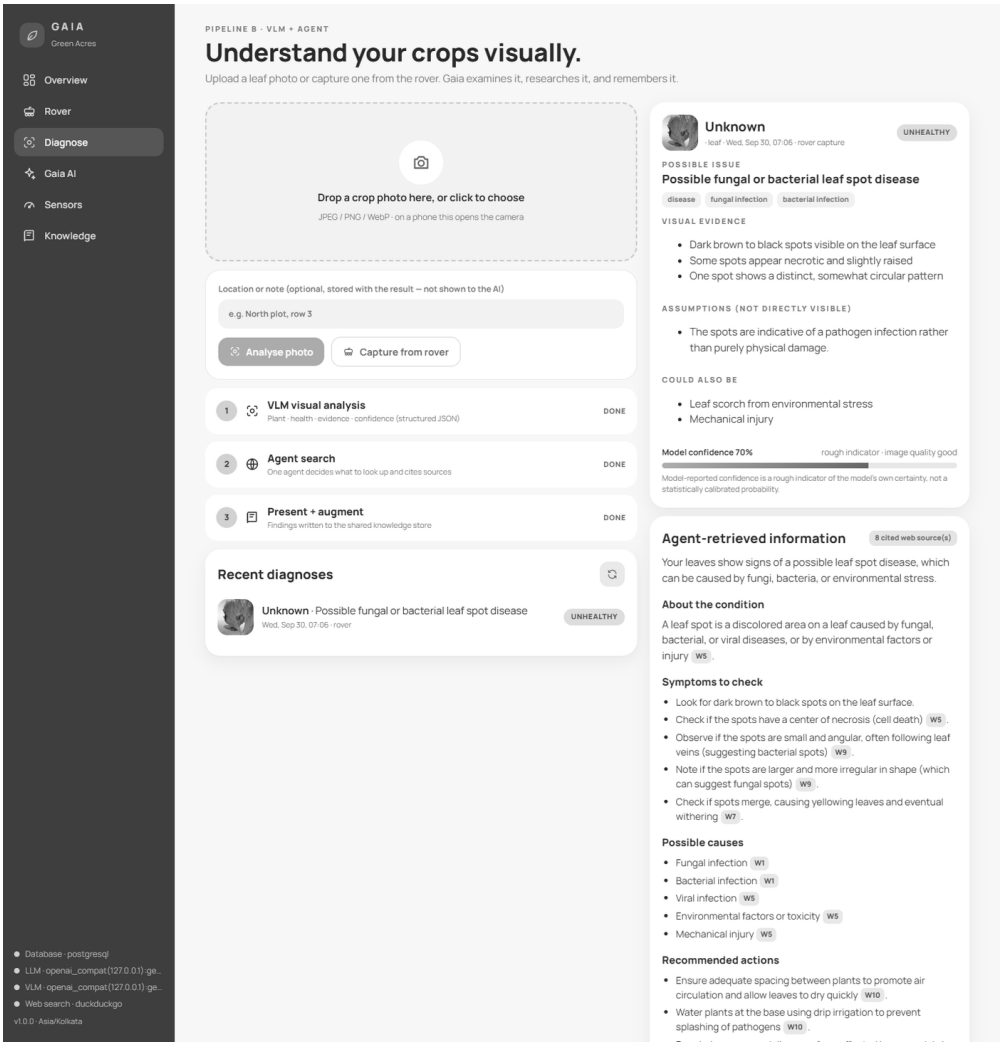


Figure 7.4: Web interface: a diagnosis result with evidence, confidence caveat and sources.

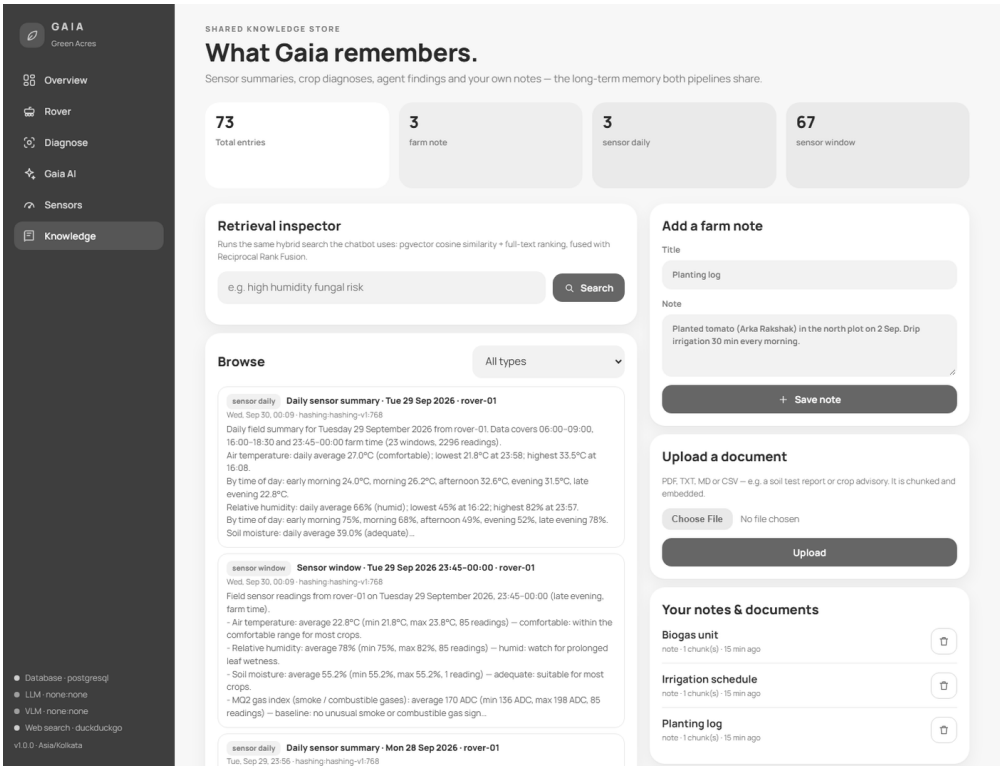


Figure 7.5: Web interface: knowledge-store inspector showing vector, lexical and fused scores.

Insert image: Live view screenshot

figures/app_live.png

Insert image: Rover tab with sensor switches

figures/app_switches.png

Figure 7.6: Mobile application: Live view and per-sensor switches.

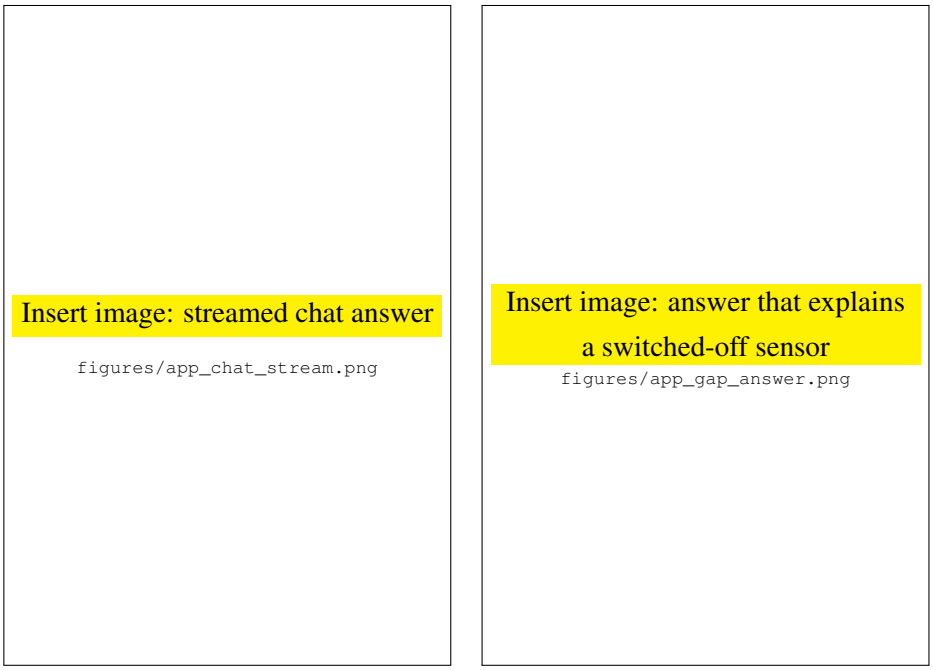


Figure 7.7: Mobile application: streamed answer and gap explanation.

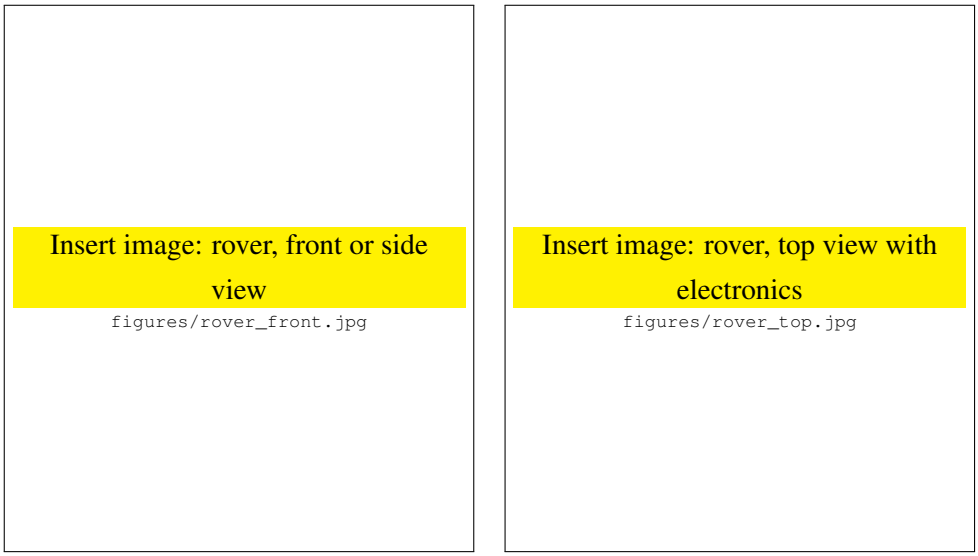


Figure 7.8: The assembled rover.

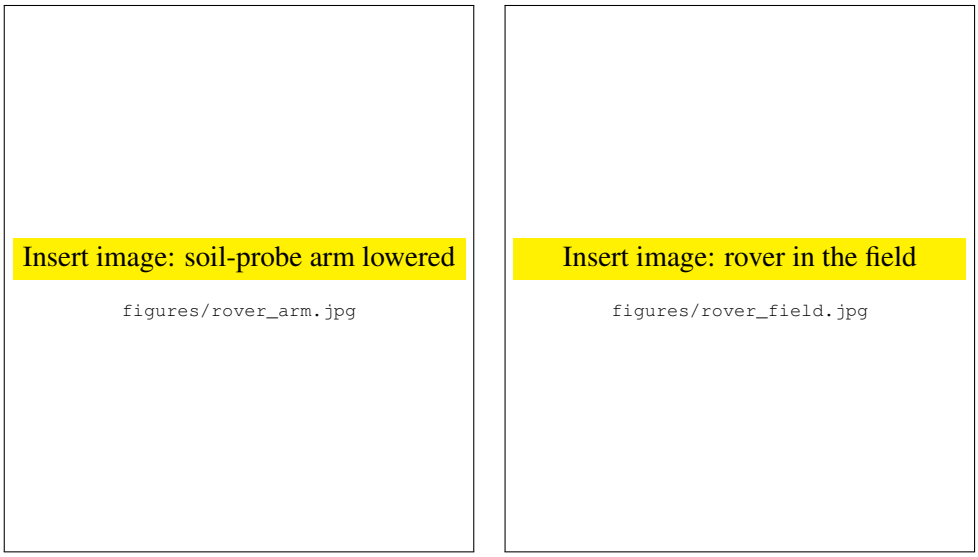


Figure 7.9: Soil-probe arm and field trial.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 CONCLUSION

This project built GAIA, a rover-assisted Generative AI system for proactive crop health monitoring, and evaluated it. The rover is deliberately a thin data-acquisition layer; the engineering effort and the evaluation concentrate on two independent Generative AI pipelines that share one knowledge store.

Pipeline A answers questions about the farmer's own field. It separates the roles of code and model: exact statistics, time resolution and citation checks are done by deterministic code, while the model plans the query and phrases the answer. On a synthetic farm with ground truth computed from the raw readings, the final configurations answered all ten evaluation questions with correct numbers, the right kind of evidence and 100 % planner accuracy. **Pipeline B** turns a photograph into a possibility-phrased, evidence-based diagnosis with a single vision-language call and one bounded research agent, and feeds the findings one way into the shared store, so that the assistant can later discuss a diagnosis together with the conditions in the field.

The battery constraint of the rover led to a feature that matters beyond power saving: every sensor can be switched on or off, the backend records the switches, readings and heartbeats, and the assistant explains gaps in the data from these records instead of guessing. Streaming chat, a Live view of the pipeline, a rover simulator, bring-up tools and demonstration data complete the system, which is verified by 96 backend and 22 application automated tests and by both firmware sketches compiling without warnings.

Table 8.1 relates the outcomes to the objectives of Section 1.5.

Table 8.1: Objectives and outcomes.

Objective	Outcome	Evidence
O1	Rover firmware, HTTP API, live video, stills	Compiled sketches; check tools
O2	Validated ingestion; windows, daily and on/off chunks; embeddings	Ingestion and PostgreSQL tests; Chapter 5
O3	Pipeline A with exact facts, hybrid retrieval, validated citations	100 % on ten questions; Chapter 6
O4	Pipeline B with one vision call, one agent, one-way write	End-to-end tests and a local-model run
O5	Per-sensor switches and gap-aware answers	Switch tests; live simulator run
O6	Flutter application with streaming and Live view	Application tests; screenshots
O7	Responsible-AI practices	Prompts, validators, tests; Chapter 4
O8	Tests, evaluation and performance figures	Chapter 6 and Chapter 7

8.2 LIMITATIONS OF THE PROPOSED SYSTEM

- **Confidence is uncalibrated.** The model's self-reported confidence is only a rough indicator. The system says so on every result and provides the metrics to measure it, but it does not correct it.
- **The vision model can be wrong.** It can misidentify a crop or a disease, especially on poor photographs; results are possibilities with evidence and look-alikes, not diagnoses.
- **Search quality varies.** The agent reads search snippets, not full web pages, and results depend on the region and the language.
- **Sensor limits.** The MQ sensors are uncalibrated and cross-sensitive and are reported as relative indices; the DHT11 has about $\pm 2\,^{\circ}\mathrm{C}$ and $\pm 5\,\%$ RH accuracy; the soil probe needs per-probe calibration.
- **No location.** Readings belong to the rover, not to a plot, because there is no GPS or odometry.
- **Small evaluation.** The evaluation uses ten questions on a synthetic farm; the vision

benchmark is an upper bound for field photographs, and results with the cloud model are reported only where they were measured.

- **Power saving needs hardware.** A switch saves energy only when the sensor's supply passes through an electronic switch; otherwise the sensor is only ignored in software. Gas sensors need a few minutes to warm up after being switched on.
- **Controller layout.** The firmware reads all sensors, including the ultrasonic one, on a single Nano; using a second Nano for the ultrasonic sensor would need an additional link to the ESP32.
- **Single farm and user.** There are no accounts, and an optional shared API key is the only access control.
- **Search latency.** The semantic search over the newest window lags by up to about 150 seconds after the window closes, although exact statistics are always current.

8.3 FUTURE WORK

- Calibrate the confidence (for example by mapping it to measured accuracy on a field-photograph set) and run a small usability study with farmers.
- For uncertain results, ask for a better photograph before searching, which is currently an open design item.
- Add a section in the application for farmers' own notes and documents (the backend already stores and retrieves them).
- Let the chat offer a button that switches a sensor on when its tip is shown.
- Track the rover's position (GPS or wheel odometry) so that readings and diagnoses belong to plots.
- Add push alerts for unusual readings, adaptive sampling that raises the rate when conditions change, and solar charging.
- Multi-tenant authentication, offline-first mobile synchronisation, and Indian-language voice interaction.
- A larger evaluation: more questions, several cloud and local models, and a field-photograph benchmark.

8.4 APPLICATIONS

- **Field scouting for small and medium farms.** Walk or drive the rover through a plot, measure conditions on demand and ask questions in plain language.
- **Early warning of disease pressure.** Link high-humidity periods in the sensor log to photographs and diagnoses taken at the same time.
- **Greenhouses and polyhouses.** The same pipeline can answer questions about temperature, humidity and gas levels in a closed space.
- **Livestock sheds and biogas units.** The gas sensors can flag unusual readings near cattle sheds or digesters.
- **Agricultural extension and education.** The system demonstrates grounded, honest generative AI, and the simulator and demonstration data run without any hardware.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[2] T. B. Brown, B. Mann, N. Ryder *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[3] Gemini Team, "Gemini: A family of highly capable multimodal models," arXiv preprint arXiv:2312.11805, 2023.

[4] G. Comanici, E. Bieber *et al.*, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," arXiv preprint arXiv:2507.06261, 2025.

[5] P. Lewis, E. Perez, A. Piktus *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[6] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv preprint arXiv:2312.10997, 2023.

[7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[8] L. Wang, C. Ma, X. Feng *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[9] Z. Ji, N. Lee, R. Frieske *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[10] S. Kadavath, T. Conerly, A. Askell, T. Henighan, D. Drain, E. Perez *et al.*, "Language models (mostly) know what they know," arXiv preprint arXiv:2207.05221, 2022.

[11] M. Xiong, Z. Hu, X. Lu, Y. Li, J. Fu, J. He, and B. Hooi, "Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs," in *International Conference on Learning Representations (ICLR)*, 2024.

[12] PEAT GmbH, "Plantix: Plant disease identification and crop advisory mobile application," https://plantix.net, accessed: 2026.

[13] K. N. Quoc, P. D. Dao, and L.-D. Quach, "LeafNet: A large-scale dataset and comprehensive benchmark for foundational vision-language understanding of plant diseases," arXiv preprint arXiv:2602.13662, 2026.

[14] E. Ranario and M. J. Earles, "Are vision-language models ready to zero-shot replace supervised classification models in agriculture?" arXiv preprint arXiv:2512.15977, 2025.

[15] K. G. Liakos, P. Busato, D. Moshou, S. Pearson, and D. Bochtis, "Machine learning in agriculture: A review," *Sensors*, vol. 18, no. 8, p. 2674, 2018.

[16] A. Kamilaris and F. X. Prenafeta-Boldú, "Deep learning in agriculture: A survey," *Computers and Electronics in Agriculture*, vol. 147, pp. 70–90, 2018.

[17] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.

[18] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," arXiv preprint arXiv:1511.08060, 2015.

[19] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018.

[20] B. Silva, L. Nunes, R. Estevão, V. Aski, and R. Chandra, "GPT-4 as an agronomist assistant? answering agriculture exams using large language models," arXiv preprint arXiv:2310.06225, 2023.

[21] A. Balaguer, V. Benara, R. L. de Freitas Cunha *et al.*, "RAG vs fine-tuning: Pipelines, tradeoffs, and a case study on agriculture," arXiv preprint arXiv:2401.08406, 2024.

[22] S. Yang, Z. Liu, and W. Mayer, "ShizishanGPT: An agricultural large language model integrating tools and resources," arXiv preprint arXiv:2409.13537, 2024.

[23] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 70, 2017, pp. 1321–1330.

[24] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[25] A. Kane, "pgvector: Open-source vector similarity search for PostgreSQL," https://github.com/pgvector/pgvector, accessed: 2026.

[26] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.

[27] J. Lee, F. Chen, S. Dua, D. Cer, M. Shanbhogue, I. Naim *et al.*, "Gemini embedding: Generalizable embeddings from gemini," arXiv preprint arXiv:2503.07891, 2025.

[28] A. Radford, J. W. Kim, C. Hallacy *et al.*, "Learning transferable visual models from natural language supervision," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 139, 2021, pp. 8748–8763.

[29] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[30] Google, "Gemini API documentation: models, structured output, embeddings and streaming," https://ai.google.dev/gemini-api/docs, accessed: 2026.

[31] Espressif Systems, "ESP32 series documentation and Arduino-ESP32 core," https://docs.espressif.com, accessed: 2026.

[32] S. Ramírez, "FastAPI framework documentation," https://fastapi.tiangolo.com, accessed: 2026.

[33] Tavily, "Tavily search API documentation," https://docs.tavily.com, accessed: 2026.

[34] Google, "Flutter documentation," https://docs.flutter.dev, accessed: 2026.

[35] WHATWG, "HTML living standard: Server-sent events," https://html.spec.whatwg.org/multipage/server-sent-events.html, accessed: 2026.

[36] G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.

APPENDIX A: PAPER PUBLICATION DETAILS

Add here the details of the paper (conference or journal name, comments of reviewers, certificate and the paper itself), the online published paper and certificates of participation in conferences, or the draft paper; or the patent filing documents; or the project event participation certificates, as applicable.

Table 8.2: Publication details (to be filled).

Item	**Details**
Paper title	
Conference or journal	
Status (draft, submitted, accepted, published)	
Reviewer comments	
Certificate or proof	

APPENDIX B: PLAGIARISM REPORT

Attach the plagiarism (similarity) check report of this project report here, generated with a good-quality similarity-check tool and verified by the guide. The similarity must be below 15 %.

APPENDIX C: REST API SUMMARY

All backend endpoints are under `/api/v1`. Endpoints other than `/health` and `POST /telemetry` require the `X-API-Key` header only when an API key is configured; telemetry needs the `X-Device-Key` header.

Table 8.3: Backend endpoints.

Area	Endpoints
Telemetry	`POST /telemetry` (readings, switches or a heartbeat), `GET /telemetry/latest`, `/telemetry/sensor-state`, `/telemetry/history`, `/telemetry/summary`, `GET /sensors`
Pipeline A	`POST /chat`, `POST /chat/stream` (SSE: session, stage, delta, final, error), `GET /chat/sessions`, `/chat/sessions/{id}`
Pipeline B	`POST /diagnoses` (image upload, returns a job), `POST /diagnoses/from-rover`, `GET /diagnoses`, `/diagnoses/{id}`, `/diagnoses/{id}/image`, `DELETE /diagnoses/{id}`
Knowledge	`GET /knowledge`, `/knowledge/search`, `/knowledge/stats`, `POST /knowledge/notes`, `POST /knowledge/documents`, `GET` and `DELETE /knowledge/documents`
Ingestion	`POST /ingest/sync`, `/ingest/reindex`, `/ingest/digest`, `GET /ingest/status`
Live view	`GET /live/state`, `GET /live/stream` (SSE of activity events)
Rover relay	`GET /rover/status`, `/rover/telemetry`, `/rover/capture`, `/rover/sensors`; `POST /rover/drive`, `/rover/stop`, `/rover/arm`, `/rover/sensors`, `/rover/telemetry/pull`
System	`GET /health`, `/config`, `/overview`